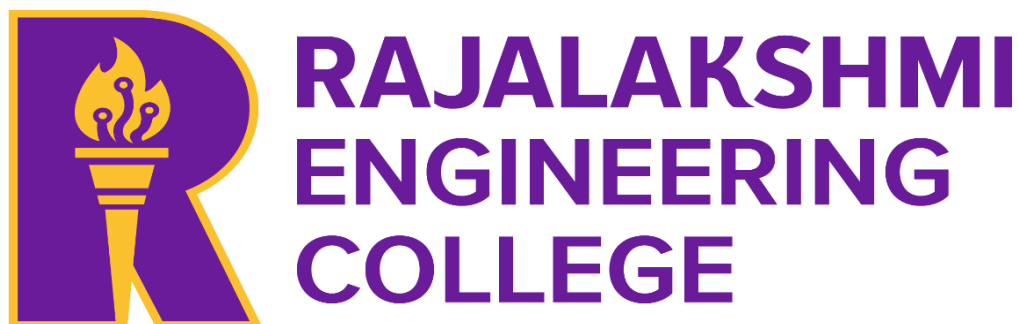


RAJALAKSHMI ENGINEERING COLLEGE

An Autonomous Institution, Affiliated to Anna University

Rajalakshmi Nagar, Thandalam – 602 105



R-2023

VII SEMESTER

B.Tech. – Artificial Intelligence & Data Science

CS23633 – CLOUD COMPUTING
LABORATORY MANUAL

NAME : DARSHINI R
REGISTER: 2116231801026
ACADEMIC YEAR:2026-2027

**BONAFIDE CERTIFICATE****NAME** **DARSHINI R****ACADEMIC YEAR** **2026-2027** **SEMESTER** **VII****BRANCH** **Artificial Intelligence and Data Science****UNIVERSITY REGISTER No.****2116231801026**

Certified that this is the bonafide record of work done by the above student in the **Cloud Computing** Laboratory during the year **2026 - 2027**

Signature of Faculty - in - Charge**Submitted for the Practical Examination held****on****Internal Examiner****External Examiner**

INDEX

S.No	Name of the Experiment	Date	Github QR	Faculty Sign
1	Private Cloud Using Openstack			
2	PaaS in Public Cloud Using Render			
3	Setting Up Hybrid Cloud Connectivity Using Cisco Packet Tracer			
4	Virtual Private Server (VPS) Provisioning and Configuration			
5	Cloud Backup and Migration using Google Drive			
6	CDN Service Selection and Configuration			

EXP NO 1	PRIVATE CLOUD USING OPENSTACK
Date: 24.06.2026	

AIM:

To create a private cloud environment by installing and configuring OpenStack (MicroStack) on a single Ubuntu system. The experiment also helps in learning how cloud resources such as virtual machines, networking, and storage are managed in a private cloud.

What is Openstack?

OpenStack is an open-source cloud platform used to build and manage cloud infrastructure. It is mainly used for Infrastructure as a Service (IaaS), where users can create and control virtual machines whenever required. It provides services for computing, networking, and storage in a cloud environment.

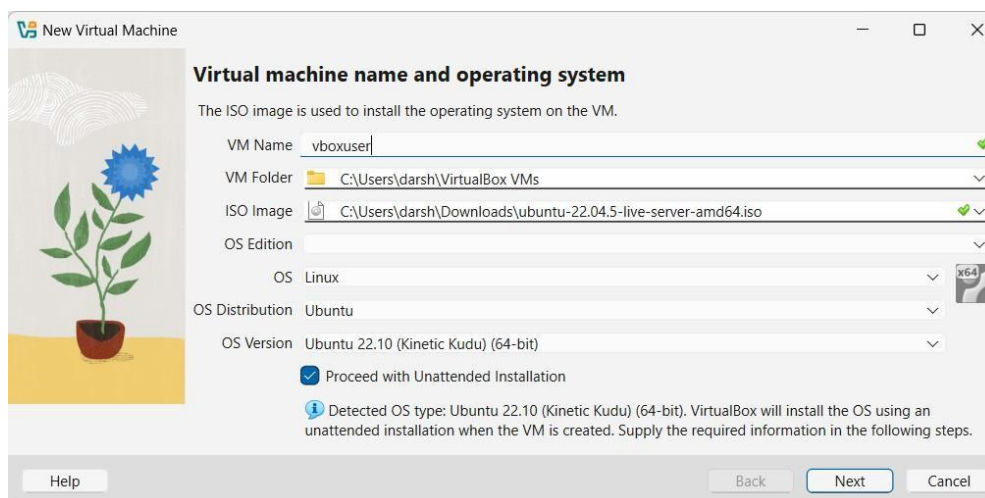
What is Microstack?

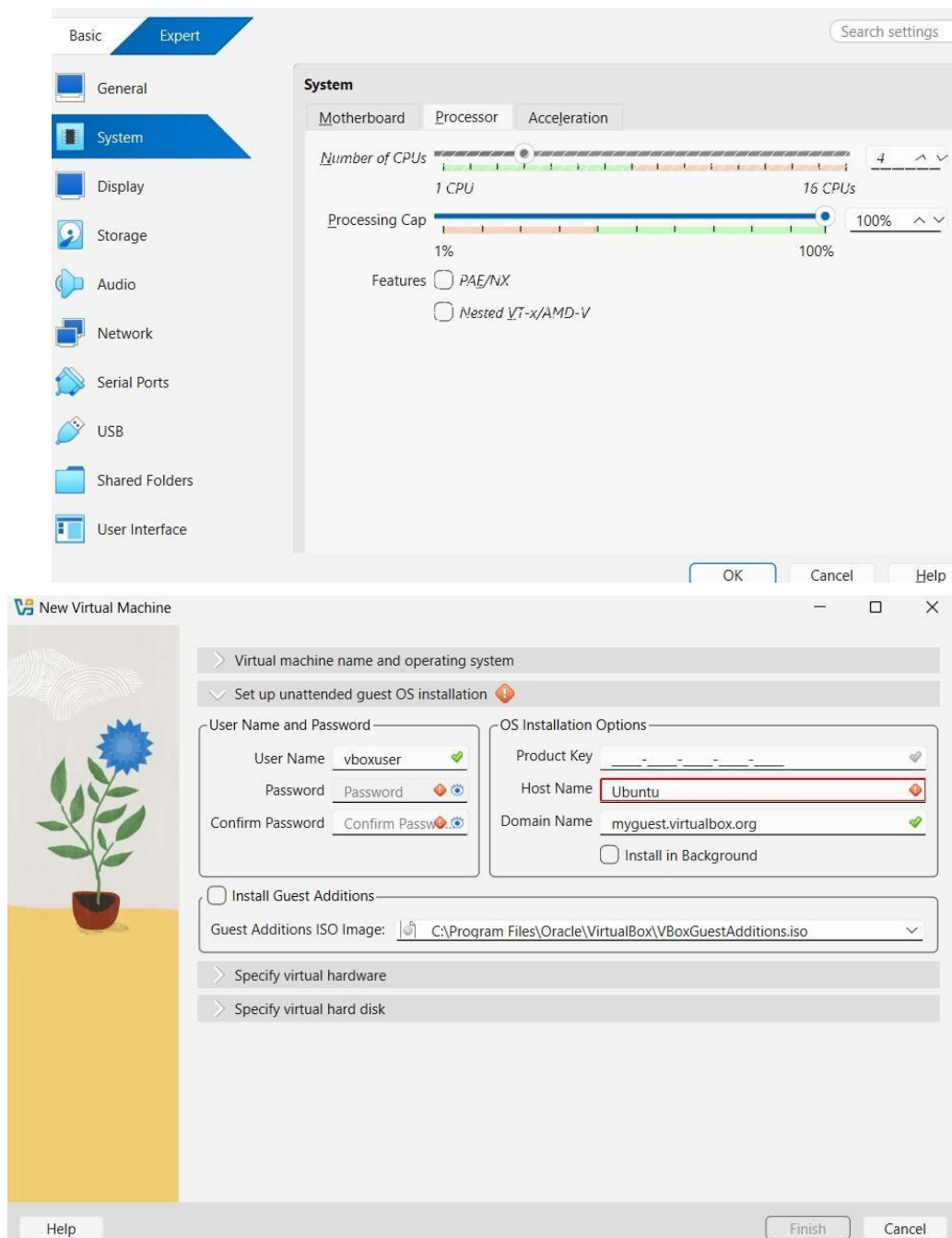
MicroStack is a lightweight version of OpenStack that runs on a single computer. It is mainly used for learning, testing, and small-scale development. Since it requires only minimal configuration, it is a good choice for beginners who want to understand how OpenStack works.

PROCEDURE:**Step 1**

Create a new Ubuntu virtual machine using Oracle VirtualBox with the following configuration:

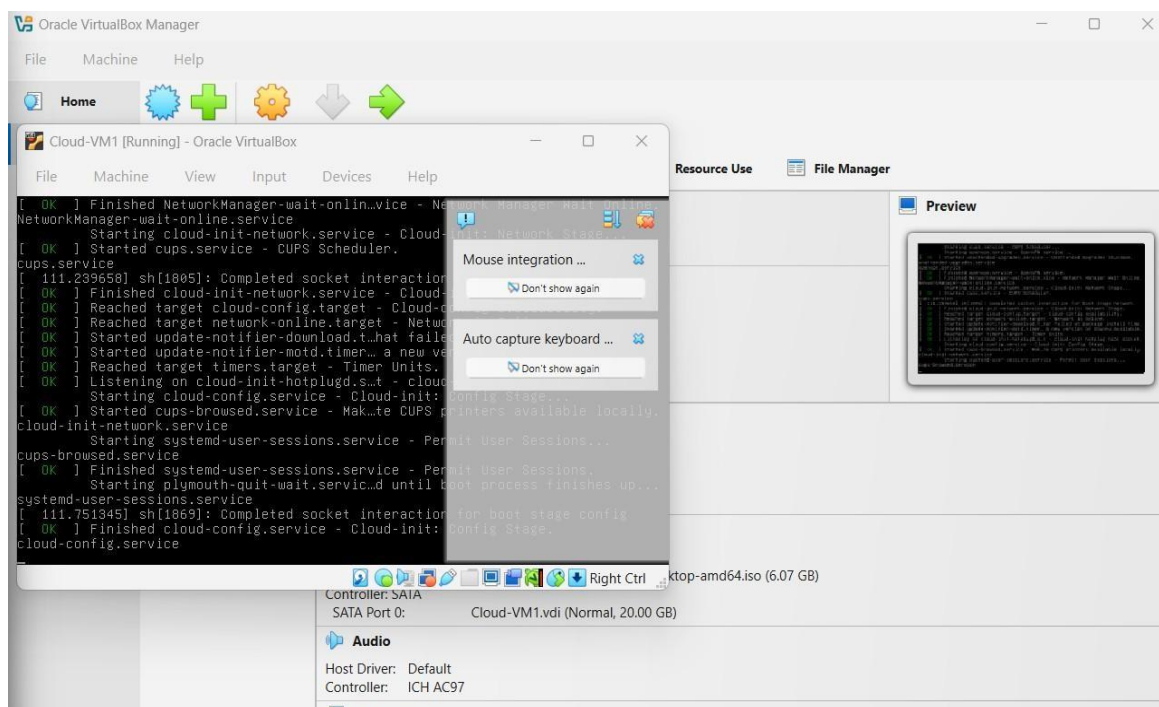
- **Memory:** 8 GB RAM
- **Processor:** 4 CPU Cores
- **Storage:** 20 GB Virtual Disk





Step 2

Install the Ubuntu operating system and log in using the username and password created during installation.



Step 3

Update the Ubuntu packages:

```
sudo apt update
```

```
sudo apt upgrade
```

Step 4

Install MicroStack (Beta channel): `sudo`

```
snap install microstack --beta
```

Step 5

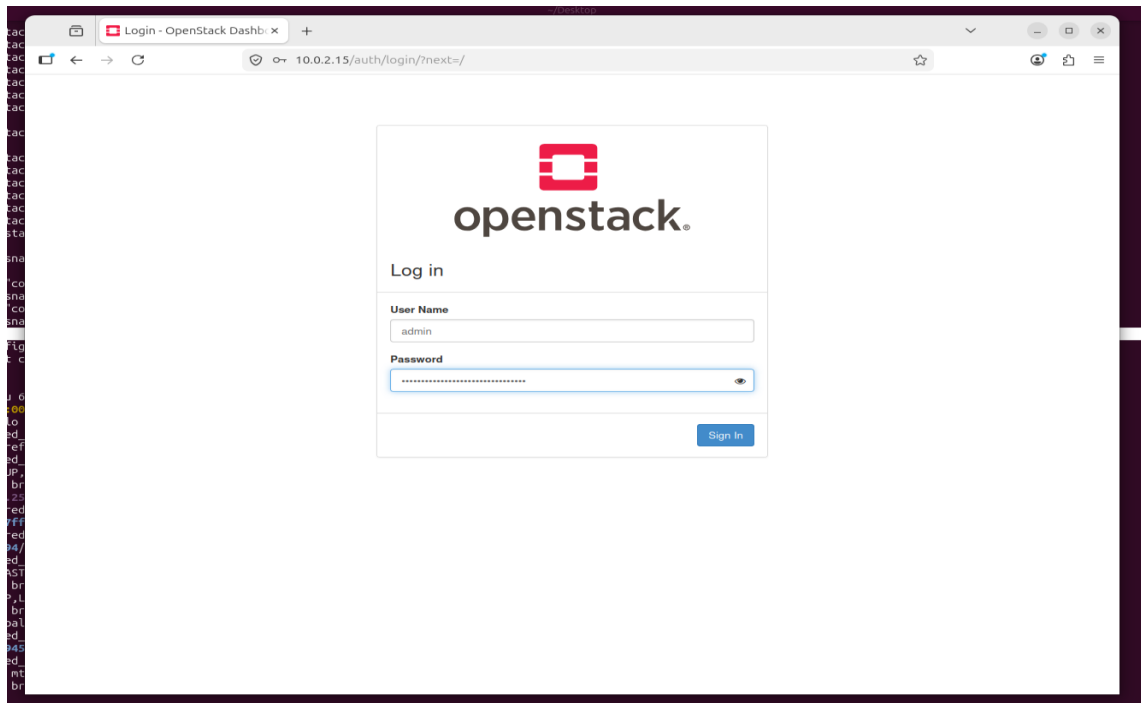
Check whether MicroStack has been installed

successfully. `snap list microstack`

Step 6

Initialize the OpenStack services automatically by executing the following

command. `sudo microstack init --auto --control`



Step 7

Verify the installed OpenStack version:

```
microstack.openstack --version
```

Step 8

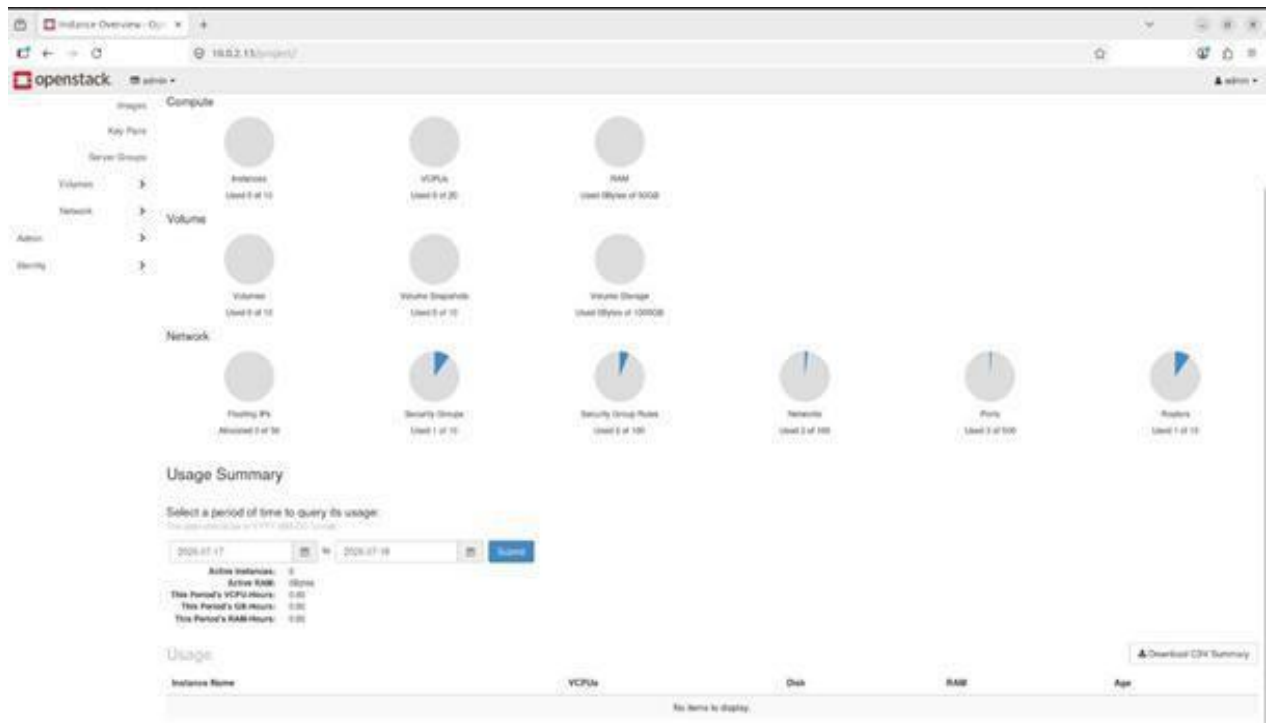
Retrieve the Keystone administrator password:

```
sudo snap get microstack config.credentials.keystone-password
```

Step 9

Open the OpenStack Horizon Dashboard in the web browser using the IP address provided after initialization and log in using:

- Username: admin
- Password: (obtained from the previous command)



CONCLUSION

OpenStack (MicroStack) was successfully installed and configured on Ubuntu, and the Horizon Dashboard was accessed .

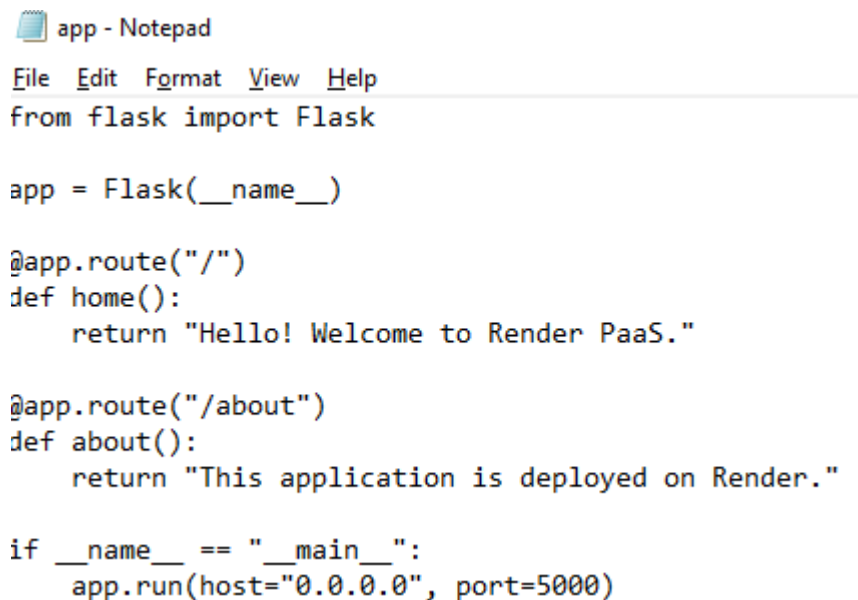
EXP NO 2	Develop a simple application to understand the concept of PAAS using Render and launch it
Date: 08.07.2026	

AIM:

To develop and deploy a simple web application using **Render**, a **Platform as a Service (PaaS)** cloud platform, and understand how cloud platforms host applications without requiring users to manage the underlying servers and infrastructure.

PROCEDURE

1. Create a project folder named **RenderHelloApp** on the local computer and create two files, **app.py** and **requirements.txt**.



```

File Edit Format View Help
from flask import Flask

app = Flask(__name__)

@app.route("/")
def home():
    return "Hello! Welcome to Render PaaS."

@app.route("/about")
def about():
    return "This application is deployed on Render."

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)

```



```

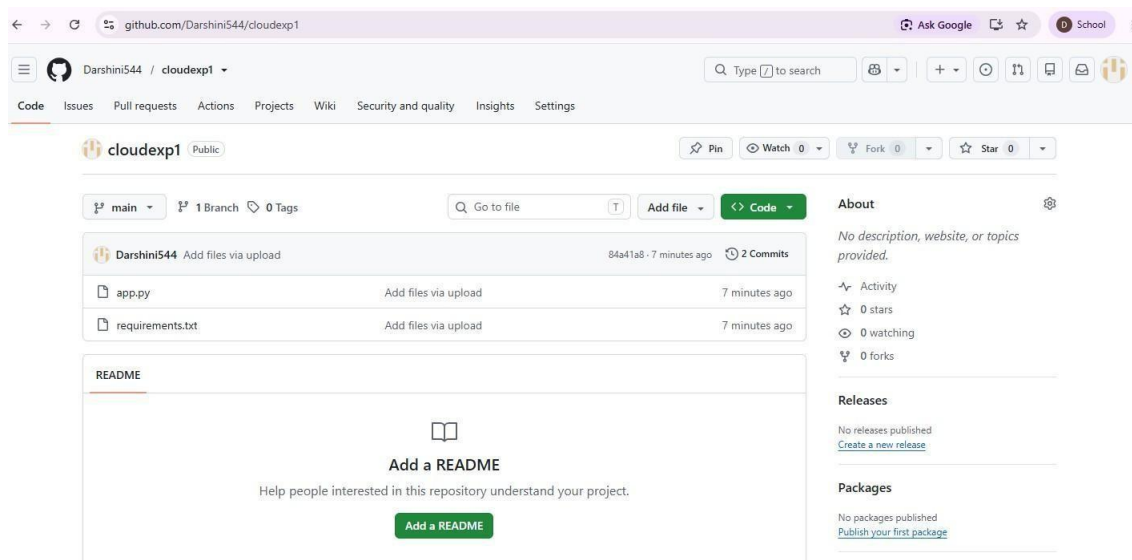
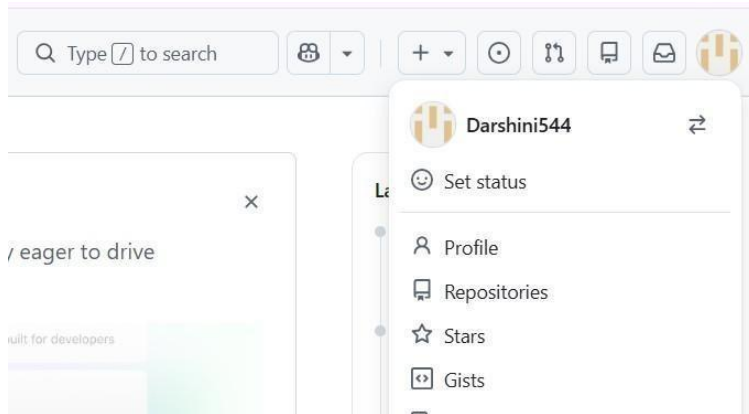
File Edit Format View Help
Flask==3.0.3
gunicorn==22.0.0

```

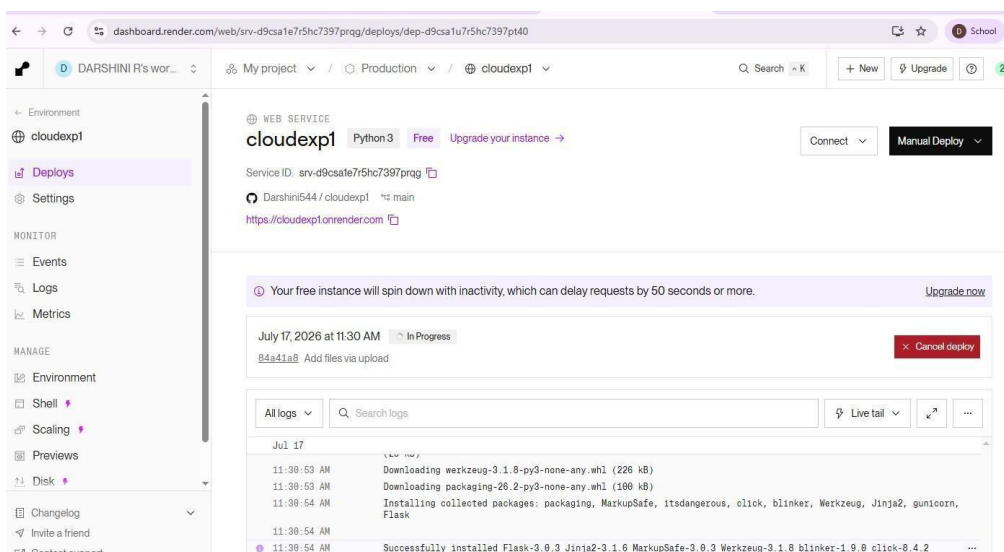
2. Write the Flask application code in **app.py** to display a welcome message on the

home page and an information message on the /about page.

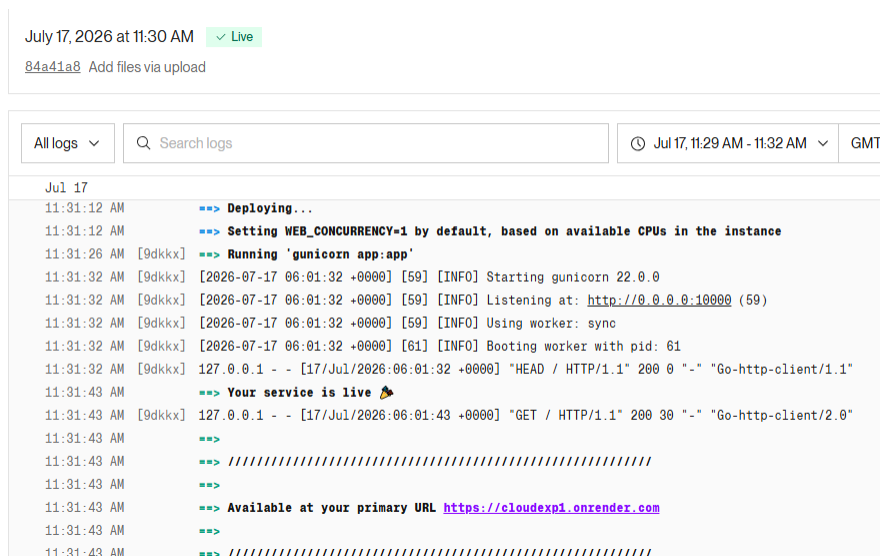
3. Add the required Python dependency, **Flask** and **Gunicorn**, in the **requirements.txt** file.
4. Sign in to **GitHub** and create a new repository named **cloudexp1** with public visibility.



5. Upload **app.py** and **requirements.txt** to the GitHub repository and click **Commit changes** to save the files.
6. Open **Render** and sign in using the GitHub account.



7. From the Render dashboard, select **Web Service** to deploy the Flask web application.
8. Connect the GitHub account to Render and select the **cloudexp1** repository containing the application files.
9. Configure the web service with the **main** branch, leave the Root Directory blank, and select **Python** as the environment.
10. Set the **Build Command** as `pip install -r requirements.txt` and the **Start Command** as `gunicorn app:app`, then select the available free instance type.
11. Click **Deploy Web Service** and wait for Render to build and deploy the application. Verify that the



deployment status shows **Live**.

12. Open the generated Render URL in a web browser and verify that the home page displays “**Hello! Welcome to Render PaaS.**” Then open the /about URL and verify the corresponding message.
13. Test both pages and capture screenshots of the **Render Dashboard, Home Page,** and **About Page** as evidence of successful deployment.



CONCLUSION

Thus, a simple Flask web application was successfully developed and deployed using **Render PaaS**. The experiment demonstrated how a PaaS platform automatically manages the server environment, software installation, deployment, and hosting, allowing applications to be accessed through a public web URL without directly managing the underlying infrastructure.

EXP NO 3	HYBRID CLOUD CONNECTIVITY USING CISCO PACKET TRACER
Date: 22.07.2026	

AIM:

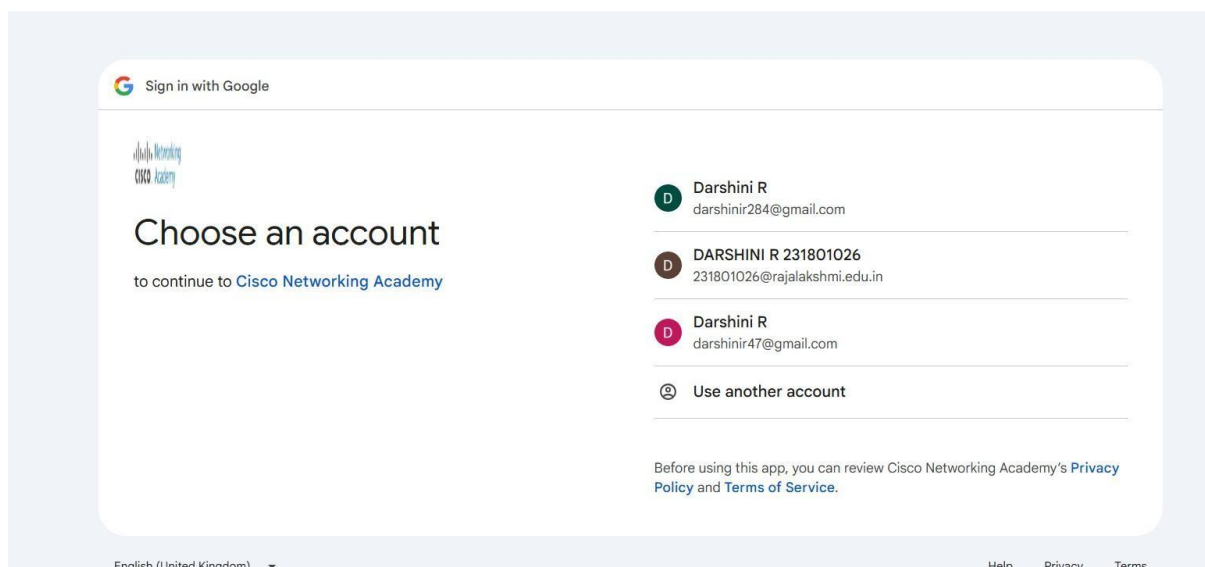
To simulate a secure hybrid cloud connection between an on-premises data center and a cloud network using routers, switches, subnets, and routing in Cisco Packet Tracer

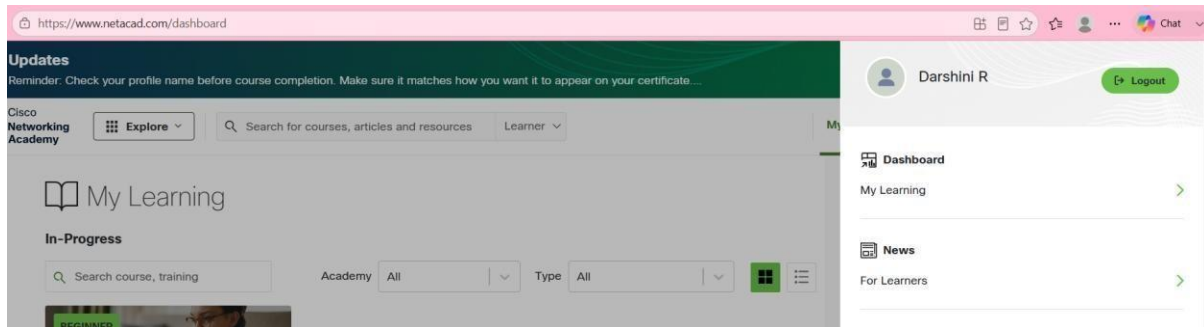
PROCEDURE:**Step 1: Open Cisco Networking Academy**

Go to: <https://www.netacad.com>

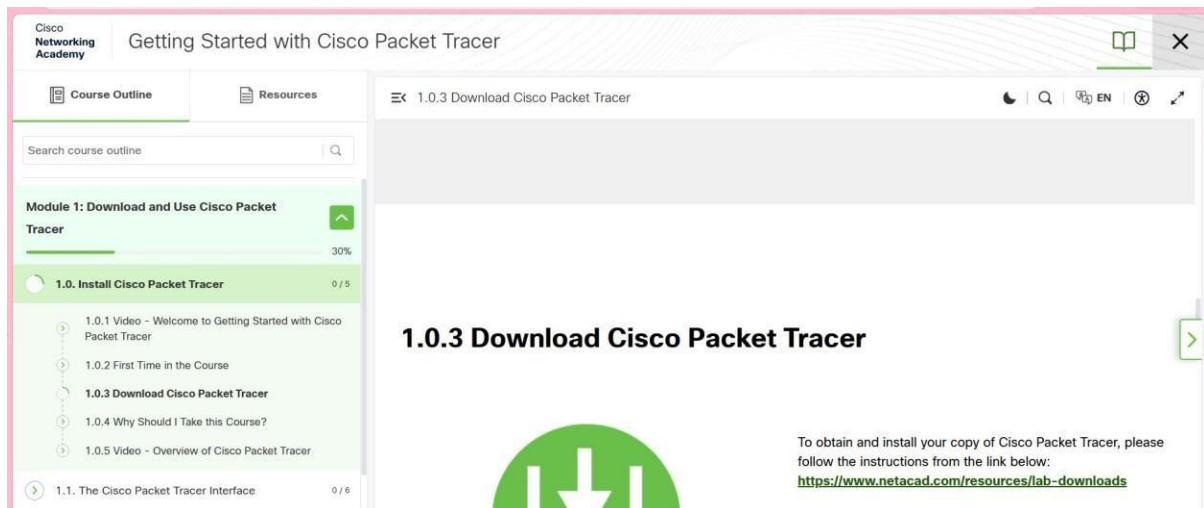
Step 2: Create a Free Account or Sign in with your Google Account

1. Click Create an account (or Sign up).
2. Enter:
 - o First Name
 - o Last Name
 - o Email Address
 - o Password
3. Verify your email if prompted.
4. Sign in.





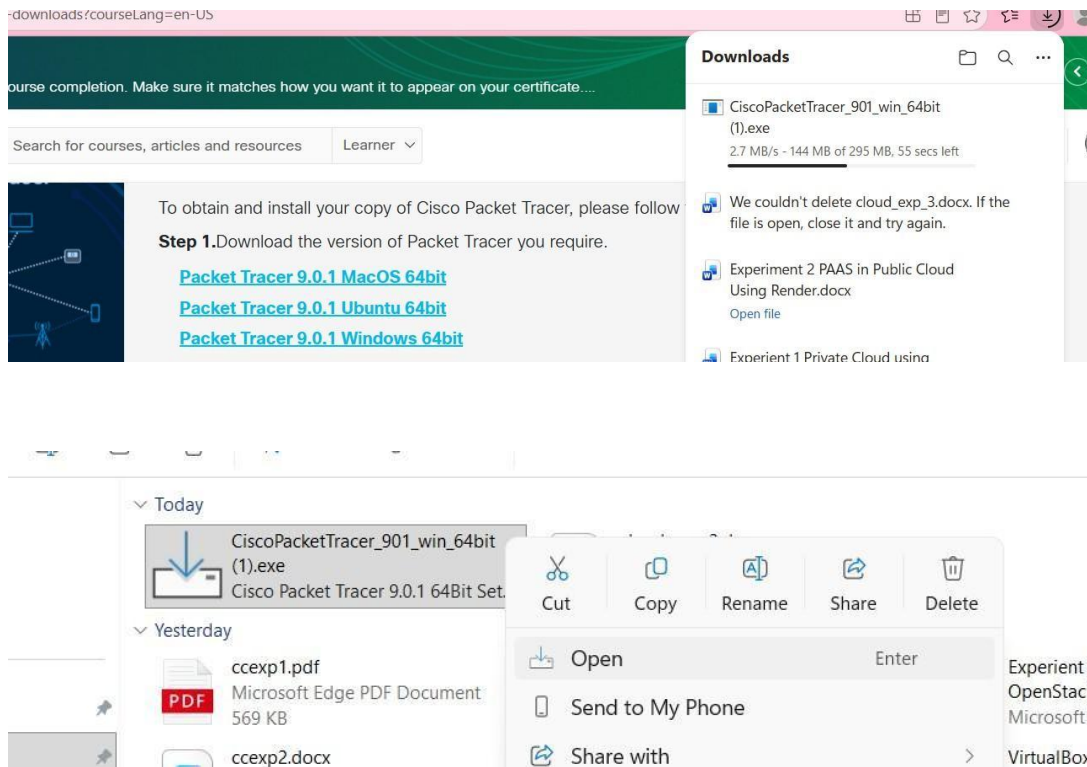
Step 3: Enroll in the Free Packet Tracer Course



Step 4: Download Packet Tracer

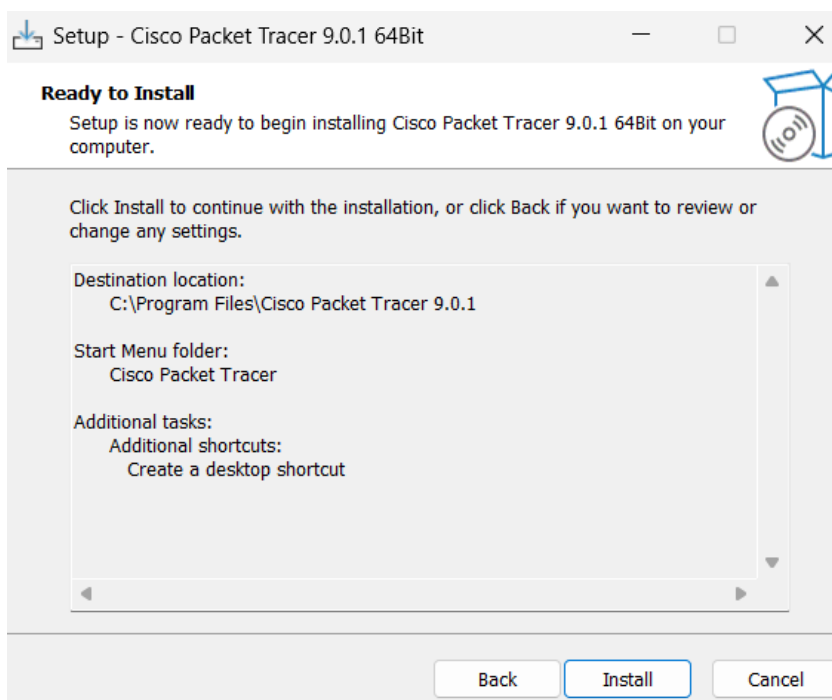
After enrolling:

1. Go to your NetAcad Dashboard.
2. Open the Getting Started with Cisco Packet Tracer course.
3. Navigate to the Resources or Downloads section.
4. Download the version for your operating system:
 - o Windows 64-bit (recommended for most PCs)
 - o macOS
 - o Linux



Step 5: Install Packet Tracer

1. Open the downloaded installer (.exe on Windows).
2. Click **Next**.
3. Accept the license agreement.
4. Click **Next** until installation completes.
5. Click **Finish**.



Step 6: Start Packet Tracer

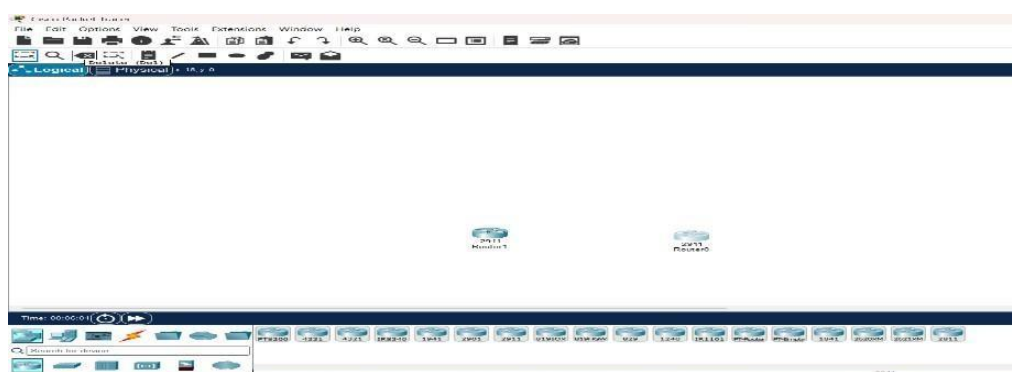
Open **Cisco Packet Tracer** from the Start Menu.

Sign in with your **Cisco NetAcad** account or sign in with your google account.

Step 7: Place Two Routers

At the **bottom-left** of Packet Tracer, you'll see several icons.

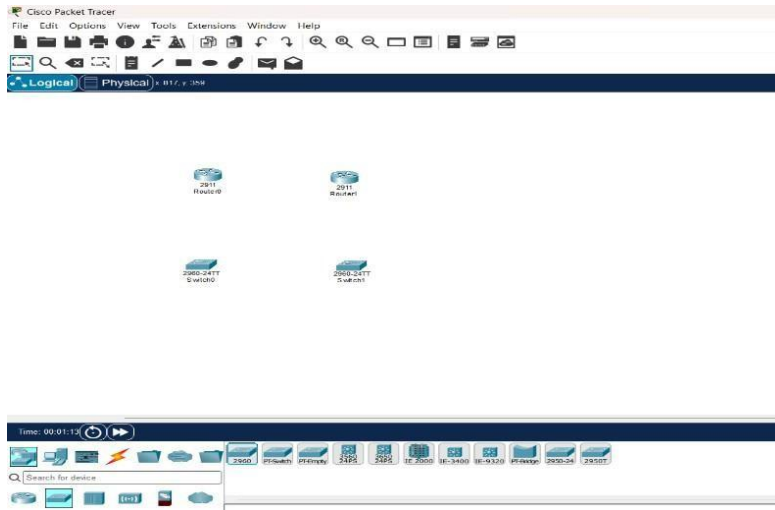
1. Click **Network Devices** (the router icon).
2. Click **Routers**.
3. Choose **2911** (or **1941** if 2911 isn't available).
4. Click once in the blank workspace to place the first router and second one



Step 8: Place Two Switches

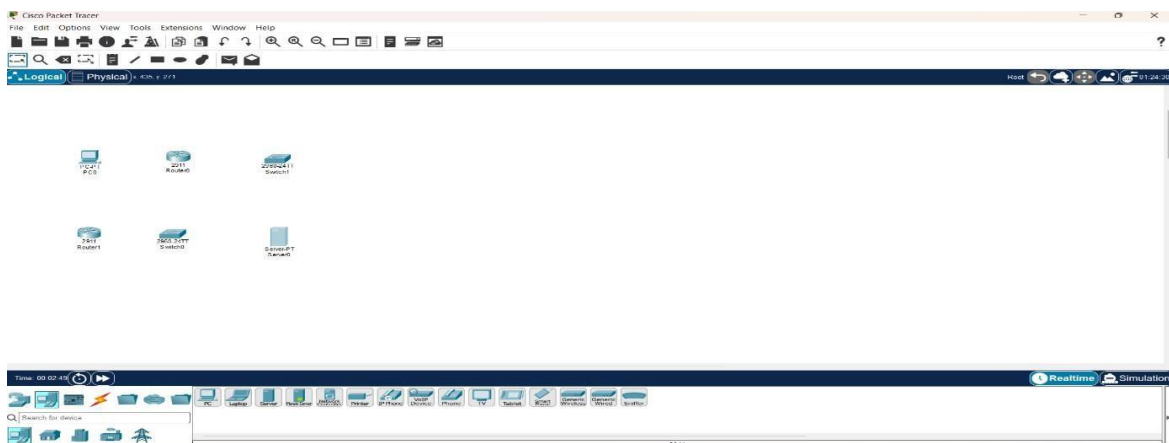
1. At the bottom-left, click **Network Devices**.

2. Click **Switches**.
3. Select **2960** (for example, **2960-24TT**).
4. Place **one switch** below **Router0**.
5. Place **another switch** below **Router1**.



Step 9: Place One PC and One Server

1. At the bottom-left, click **End Devices** (computer icon).
2. Select **PC**.
3. Place it to the **left of Switch0**.
4. Select **Server**.
5. Place it to the **right of Switch1**.



Step 10: Connect the Devices

At the bottom-left, click the **Connections** icon (lightning bolt).

Choose **Copper Straight-Through** (usually the first black cable).

Connection 1: PC to Switch0

1. Click PC.
2. Select **FastEthernet0**.
3. Click **Switch0**.
4. Select **FastEthernet0/1** (or any free FastEthernet port).

Connection 2: Switch0 to Router0

Using the same **Copper Straight-Through** cable:

1. Click **Switch0**.
2. Select **FastEthernet0/2**.
3. Click **Router0**.
4. Select **GigabitEthernet0/0**.

Connection 3: Server to Switch1

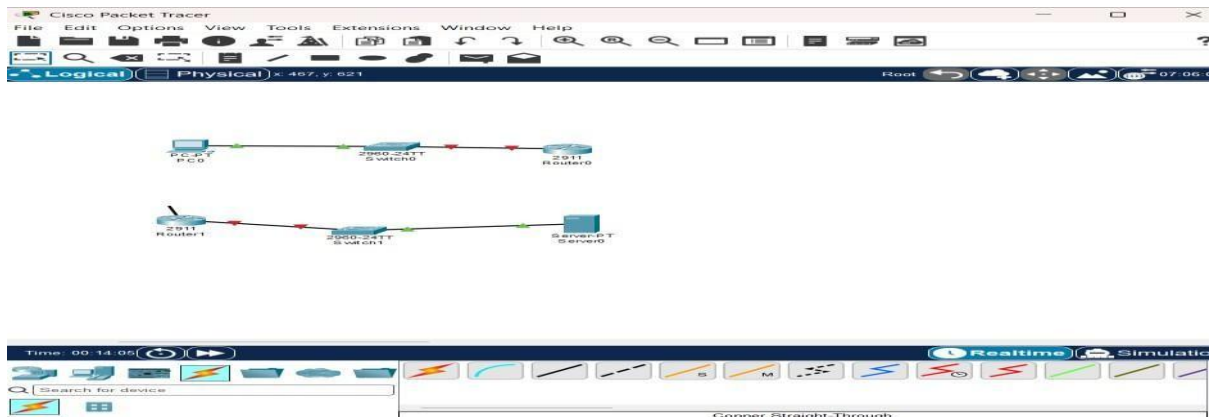
Again use **Copper Straight-Through**:

1. Click **Server**.
2. Select **FastEthernet0**.
3. Click **Switch1**.
4. Select **FastEthernet0/1**.

Connection 4: Switch1 to Router1

Again use **Copper Straight-Through**:

1. Click **Switch1**.
2. Select **FastEthernet0/2**.
3. Click **Router1**.
4. Select **GigabitEthernet0/0**.



Step 10.1: Connect Router0 to Router1

Now connect the two

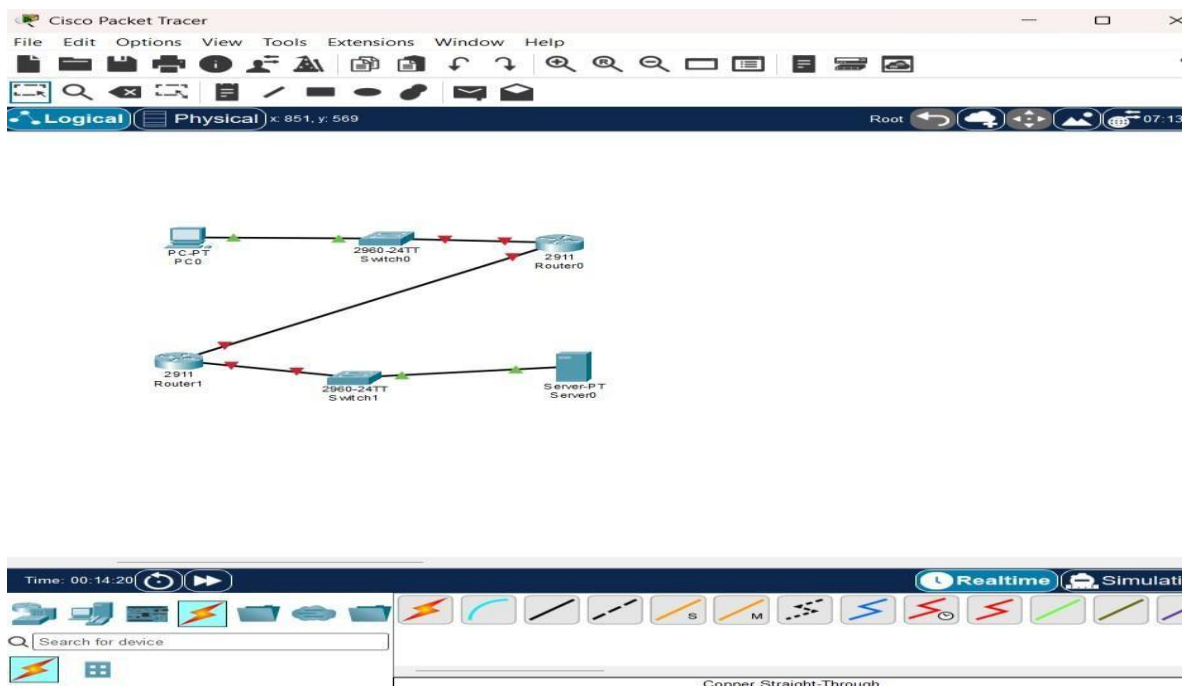
routers. **Option A**

(Recommended) Use

Copper Cross-Over.

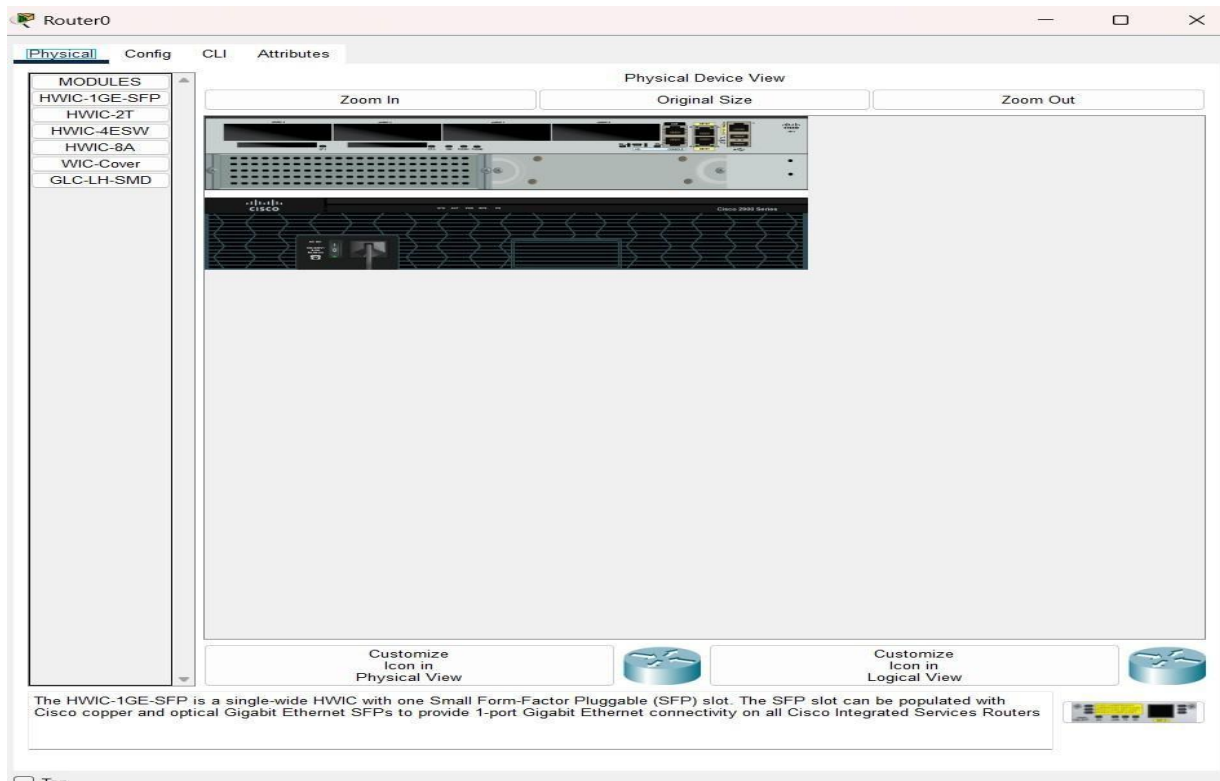
1. Click the **Connections** icon.
2. Select **Copper Cross-Over**.
3. Click Router0.
4. Select **GigabitEthernet0/1**.
5. Click **Router1**.
6. Select **GigabitEthernet0/1**.

Your topology should now look like this:



Step 10.2: Click the Router

In your workspace, **left-click Router0** once.



1. Click the CLI tab.

2. If it asks:

Continue with configuration dialog?

[yes/no]: Type: no Press **Enter**. When

you see: Router>

```

Router0
Physical Config CLI Attributes
IOS Command Line Interface

smart init is sizing iomem
TYPE MEMORY_REQ
Onboard devices 6
buffer pools 0x022F6000
-----
TOTAL: 0x022F6000
Rounded IOMEM up to: 36Mb.
Using 6 percent iomem. [36Mb/512Mb]

Restricted Rights Legend

Use, duplication, or disclosure by the Government is
subject to restrictions as set forth in subparagraph
(c) of the Commercial Computer Software - Restricted
Rights clause at FAR sec. 52.227-19 and subparagraph
(c) (1) (i) of the Rights in Technical Data and Computer
Software clause at DFARS sec. 252.227-7013.

Cisco Systems, Inc.
170 West Tasman Drive
San Jose, California 95134-1706

Cisco IOS Software, C2900 Software (C2900-UNIVERSALK9-M), Version 15.1(4)M5, RELEASE SOFTWARE (fc2) Technical
Support: http://www.cisco.com/techsupport
Copyright (c) 1986-2007 by Cisco Systems, Inc.
Compiled Wed 18-Jul-07 04:52 by pt_team
Image text-base: 0x2100F918, data-base: 0x24729040

This product contains cryptographic features and is subject to United
States and local country laws governing import, export, transfer and
use. Delivery of Cisco cryptographic products does not imply
third-party authority to import, export, distribute or use encryption.
Importers, exporters, distributors and users are responsible for
compliance with U.S. and local country laws. By using this product you
agree to comply with applicable laws and regulations. If you are unable
to comply with U.S. and local laws, return this product immediately.

A summary of U.S. laws governing Cisco cryptographic products may be found at:
http://www.cisco.com/wml/export/crypto/ctool/stgrg.html
If you require further assistance please contact us by sending email to
export@cisco.com.

Cisco CISC02911/K9 (revision 1.0) with 491520K/32768K bytes of memory.
Processor board ID FTX152400K5
3 Gigabit Ethernet interfaces
DRAM configuration is 64 bits wide with parity disabled.
256K bytes of non-volatile configuration memory.
249856K bytes of ATA System CompactFlash 0 (Read/Write)

--- System Configuration Dialog ---

Would you like to enter the initial configuration dialog? [yes/no]: |
Copy Paste

```

Step 10.3: Configure the LAN Interface (Connected to Switch0)

At the Router(config)# prompt, type these commands **one at a time**, pressing **Enter** after each line.

interface

gigabitEth

ernet0/0 You should

see:

Router(co

nfig-if)#

Now type:

ip address 192.168.1.1 255.255.255.0

Press

Enter.

Now

type:

no shutdown

You should see a message similar to: Now

```

Cisco CISC02911/K9 (revision 1.0) with 491520K/32768K bytes of memory.
Processor board ID FTX152400KS
3 Gigabit Ethernet interfaces
DRAM configuration is 64 bits wide with parity disabled.
255K bytes of non-volatile configuration memory.
249856K bytes of ATA System CompactFlash 0 (Read/Write)

--- System Configuration Dialog ---

Would you like to enter the initial configuration dialog? [yes/no]: no

Press RETURN to get started!

Router>configure terminal
      ^
% Invalid input detected at '^' marker.

Router>enable
Router#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#ip address 192.168.1.1 255.255.255.0
      ^
% Invalid input detected at '^' marker.

Router(config)#no shutdown
      ^
% Invalid input detected at '^' marker.

Router(config)#Router0
      ^
% Invalid input detected at '^' marker.

Router(config)#prompt
      ^
% Invalid input detected at '^' marker.

Router(config)##prompt
      ^
% Invalid input detected at '^' marker.

Router(config)#interface gigabitEthernet0/0
Router(config-if)#ip address 192.168.1.1 255.255.255.0
Router(config-if)#no shutdown

Router(config-if)#
%LINK-5-CHANGED: Interface GigabitEthernet0/0, changed state to up
%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0, changed state to up

```

Step 10.4: Configure the Router-to-Router Interface

Now type:

```
interface    gigabitEthernet0/1
```

Then:

```
ip address 10.10.10.1 255.255.255.252
```

Then:

```
no shutdown
```

Again, wait for the interface to come up.

Finally, type:

```
exit
```

You should now be back at:

```
Router(config)#
```

```

Router1(config-if)#no shutdown
Router(config-if)#
%LINK-5-CHANGED: Interface GigabitEthernet0/0, changed state to up
%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0, changed state to up
Router(config-if)#exit
Router(config)#interface gigabitEthernet0/1
Router(config-if)#ip address 10.10.10.1 255.255.255.252
Router(config-if)#no shutdown

Router(config-if)#
%LINK-5-CHANGED: Interface GigabitEthernet0/1, changed state to up

```

Step 10.5: Verify the Configuration

Type:

do show ip interface brief

Since you're in configuration mode, use **do** before the show command.

You should see something similar to:

```

Router#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#do show ip interface brief
Interface                IP-Address      OK? Method Status        Protocol
GigabitEthernet0/0       192.168.1.1     YES manual up            up
GigabitEthernet0/1       10.10.10.1      YES manual up            up
GigabitEthernet0/2       unassigned      YES unset  administratively down down
Vlan1                    unassigned      YES unset  administratively down down
Router(config)#ip route 10.0.0.0 255.255.255.0 10.10.10.2
Router(config)#exit
Router#
%SYS-5-CONFIG_I: Configured from console by console

Router#copy running-config startup-config
Destination filename [startup-config]?
Building configuration...
[OK]
Router#

```

Next Step: Configure Router1

Click **Router1** → **CLI**.

If it says:

Press RETURN to get started!

Press **Enter**.

If it asks:

Continue with configuration dialog? [yes/no]:

Type:

no

Then enter these commands **one by one**: enable

configure terminal

interface gigabitEthernet0/0

ip address 10.0.0.1 255.255.255.0

no shutdown

exit

interface gigabitEthernet0/1

ip address

```
Router>enable
Router#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#interface gigabitEthernet0/0
Router(config-if)#ip address 10.0.0.1 255.255.255.0
Router(config-if)#no shutdown

Router(config-if)#
%LINK-5-CHANGED: Interface GigabitEthernet0/0, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0, changed state to up

Router(config-if)#exit
Router(config)#
Router(config)#interface gigabitEthernet0/1
Router(config-if)#ip address 10.10.10.2 255.255.255.252
Router(config-if)#no shutdown

Router(config-if)#
%LINK-5-CHANGED: Interface GigabitEthernet0/1, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/1, changed state to up

Router(config-if)#exit
Router(config)#
Router(config)#do show ip interface brief

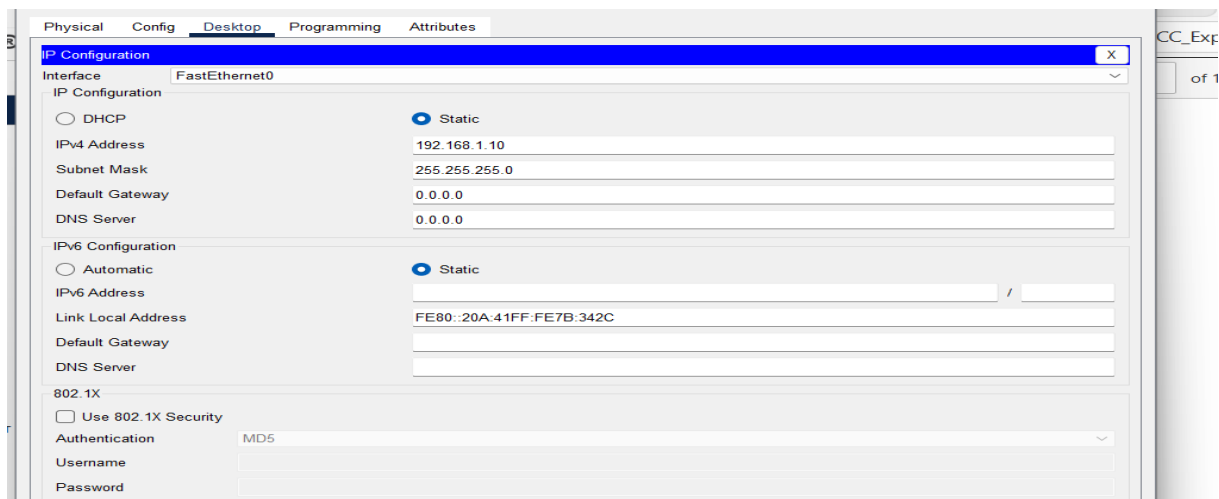
```

Interface	IP-Address	OK?	Method	Status	Protocol
GigabitEthernet0/0	10.0.0.1	YES	manual	up	up
GigabitEthernet0/1	10.10.10.2	YES	manual	up	up
GigabitEthernet0/2	unassigned	YES	unset	administratively down	down
Vlan1	unassigned	YES	unset	administratively down	down

```
Router(config)#
```

Step 10.6: Configure the PC

1. Click PC.
2. Select **Desktop**.
3. Click **IP Configuration**.



Step 10.7: Configure the Server

1. Click Server.

2. Select **Desktop**.

3. Click **IP Configuration**.

The screenshot shows the 'Server0' configuration window with the 'Desktop' tab selected. The 'IP Configuration' section is expanded, showing the following settings:

- IP Configuration:**
 - ☐ DHCP
 - ☒ Static
 - IPv4 Address: 10.0.0.10
 - Subnet Mask: 255.255.255.0
 - Default Gateway: 10.0.0.1
 - DNS Server: 0.0.0.0
- IPv6 Configuration:**
 - ☐ Automatic
 - ☒ Static
 - IPv6 Address: (empty)
 - Link Local Address: FE80::290:CFF:FEDD:9749
 - Default Gateway: (empty)
 - DNS Server: (empty)
- 802.1X:**
 - ☐ Use 802.1X Security
 - Authentication: MD5
 - Username: (empty)
 - Password: (empty)

At the bottom left, there is a 'Top' button.

Step 11: Configure Static Routes

A. Configure Static Route on Router0

1. Click **Router0**.

2. Go to the CLI tab.

3. If you're at Router#, type:

configure terminal

Now enter:

```
ip route 10.0.0.0 255.255.255.0 10.10.10.2
```

Exit configuration mode: exit

Save the configuration: copy running-config startup-config

When prompted: Destination filename [startup-config]? Just press **Enter**.

B. Configure Static Route on Router1

1. Click **Router1**.

2. Open the CLI.

At the Router# prompt, type: configure terminal

Now enter: ip route 192.168.1.0 255.255.255.0 10.10.10.1

Exit: exit

Save the configuration: copy running-config startup-config When prompted for the filename, simply press **Enter**.

```
Router#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#do show ip interface brief
Interface          IP-Address      OK? Method Status      Protocol
GigabitEthernet0/0  192.168.1.1     YES manual up          up
GigabitEthernet0/1  10.10.10.1      YES manual up          up
GigabitEthernet0/2  unassigned      YES unset  administratively down down
Vlan1               unassigned      YES unset  administratively down down
Router(config)#ip route 10.0.0.0 255.255.255.0 10.10.10.2
Router(config)#exit
Router#
%SYS-5-CONFIG_I: Configured from console by console

Router#copy running-config startup-config
Destination filename [startup-config]?
Building configuration...
[OK]
Router#
```

C. Verify the Route

Type: show ip route

You should see a line similar to: S 192.168.1.0/24 [1/0] via 10.10.10.1 The

S indicates that the static route has been added successfully.

```
Building configuration...
[OK]
Router#show ip route
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2, E - EGP
       i - IS-IS, L1 - IS-IS level-1, L2 - IS-IS level-2, ia - IS-IS inter area
       * - candidate default, U - per-user static route, o - ODR
       P - periodic downloaded static route

Gateway of last resort is not set

    10.0.0.0/8 is variably subnetted, 3 subnets, 3 masks
S       10.0.0.0/24 [1/0] via 10.10.10.2
C       10.10.10.0/30 is directly connected, GigabitEthernet0/1
L       10.10.10.1/32 is directly connected, GigabitEthernet0/1
C       192.168.1.0/24 is variably subnetted, 2 subnets, 2 masks
C       192.168.1.0/24 is directly connected, GigabitEthernet0/0
L       192.168.1.1/32 is directly connected, GigabitEthernet0/0
Router#
```

Copy

Paste

Now let's verify Router0

On **Router0**, at the Router# prompt, type: show ip route You should see a line like: S 10.0.0.0/24 [1/0] via 10.10.10.2

If you don't see it, add the route:

configure terminal ip route 10.0.0.0 255.255.255.0 10.10.10.2

end copy running-config startup-config

Press **Enter** when it asks for the destination filename.

```
Router#
%SYS-5-CONFIG_I: Configured from console by console

Router#copy running-config startup-config
Destination filename [startup-config]?
Building configuration...
[OK]
Router#show ip route
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2, E - EGP
       i - IS-IS, L1 - IS-IS level-1, L2 - IS-IS level-2, ia - IS-IS inter area
       * - candidate default, U - per-user static route, o - ODR
       P - periodic downloaded static route

Gateway of last resort is not set

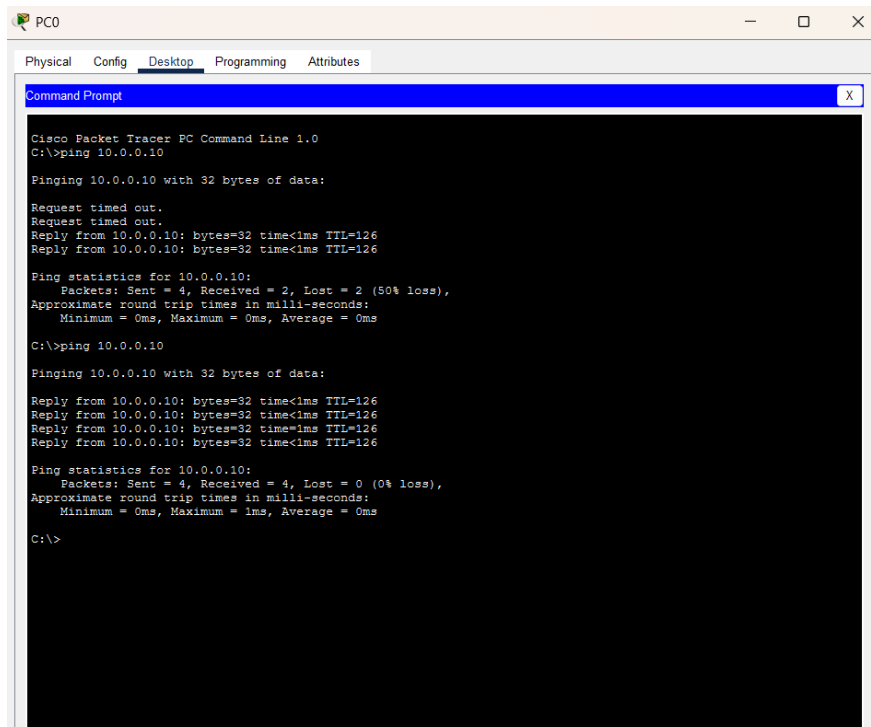
    10.0.0.0/8 is variably subnetted, 4 subnets, 3 masks
    U       10.0.0.0/24 is directly connected, GigabitEthernet0/0
    U       10.0.0.1/32 is directly connected, GigabitEthernet0/0
    U       10.10.10.0/30 is directly connected, GigabitEthernet0/1
    L       10.10.10.2/32 is directly connected, GigabitEthernet0/1
    S       192.168.1.0/24 [1/0] via 10.10.10.1

Router#
```

Step 12: Test Connectivity

From the PC

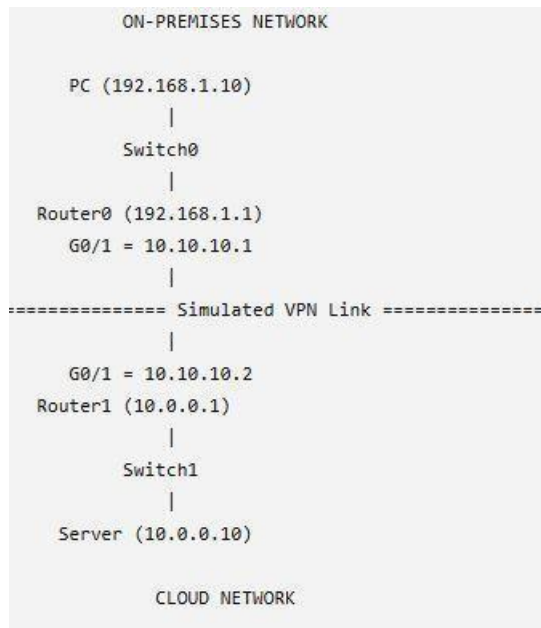
1. Click PC.
2. Go to **Desktop** → **Command Prompt**.
3. Type: ping 10.0.0.10
4. Type one more time: ping 10.0.0.10



Experiment Successfully Completed!

You have successfully simulated **Hybrid Cloud Connectivity**.

Architecture Diagram of Simulation



Result:

A hybrid cloud network was successfully simulated using Cisco Packet Tracer. The on-premises network and cloud network were interconnected through routers configured with static routing. Communication between the PC and the cloud server was verified successfully using the ping command, demonstrating end-to-end connectivity.

EXP NO 4	CDN Service Selection and Configuration
Date: 05.08.2026	

AIM:

To provision a Virtual Private Server (VPS) using VirtualBox, install Ubuntu Server, configure network settings (IP Address, Subnet Mask, Gateway, DNS), and enable SSH remote access.

What is VPS?

A Virtual Private Server (VPS) is a software-based server created using virtualization technology. It operates independently with its own operating system, storage, memory, and network configuration while sharing the physical hardware with other virtual machines. Virtualization allows efficient utilization of resources and forms the foundation of cloud computing platforms.

Network configuration parameters include:

IP Address

A unique address assigned to a device on a network.

Subnet Mask

Determines the network and host portions of an IP address.

Gateway

Provides access to networks outside the local network.

DNS (Domain Name System)

Converts domain names into IP addresses.

SSH (Secure Shell)

A secure protocol used for remote administration of Linux systems.

PROCEDURE:

1. Open the Ubuntu terminal and verify the internet connection using the ping google.com command.

```

valid_lft forever preferred_lft forever
vboxuser@Ubuntu:~/Desktop$ ping google.com
PING google.com (142.251.221.206) 56(84) bytes of data.
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=1 ttl=64 time=35.8 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=2 ttl=64 time=36.1 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=3 ttl=64 time=64.2 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=4 ttl=64 time=51.4 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=5 ttl=64 time=60.5 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=6 ttl=64 time=239 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=7 ttl=64 time=256 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=8 ttl=64 time=93.1 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=9 ttl=64 time=48.6 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=10 ttl=64 time=34.1 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=11 ttl=64 time=55.8 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=12 ttl=64 time=51.9 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=13 ttl=64 time=33.4 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=14 ttl=64 time=149 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=15 ttl=64 time=246 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=16 ttl=64 time=64.7 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=17 ttl=64 time=81.8 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=18 ttl=64 time=48.2 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=19 ttl=64 time=44.5 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=20 ttl=64 time=264 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=21 ttl=64 time=122 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=22 ttl=64 time=50.8 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=23 ttl=64 time=48.9 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=24 ttl=64 time=52.6 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=25 ttl=64 time=34.0 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=26 ttl=64 time=56.1 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=27 ttl=64 time=60.1 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=28 ttl=64 time=148 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=29 ttl=64 time=45.3 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=30 ttl=64 time=64.6 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=31 ttl=64 time=277 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=32 ttl=64 time=42.7 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=33 ttl=64 time=36.0 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=34 ttl=64 time=241 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=35 ttl=64 time=301 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=36 ttl=64 time=37.5 ms
64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=37 ttl=64 time=58.2 ms

```

2. Check the network routing information using the ip route command.

```

valid_lft forever preferred_lft forever
vboxuser@Ubuntu:~/Desktop$ ip route
default via 10.0.2.2 dev enp0s3 proto dhcp src 10.0.2.15 metric 100
10.0.2.0/24 dev enp0s3 proto kernel scope link src 10.0.2.15 metric 100
10.20.20.0/24 dev br-ex proto kernel scope link src 10.20.20.1
vboxuser@Ubuntu:~/Desktop$ cat /etc/resolv.conf

```

3. Verify the DNS (Domain Name System) configuration of the system using the cat /etc/resolv.conf command.

```
vboxuser@Ubuntu:~/Desktop$ cat /etc/resolv.conf
# This is /run/systemd/resolve/stub-resolv.conf managed by man:systemd-resolved(8).
# Do not edit.
#
# This file might be symlinked as /etc/resolv.conf. If you're looking at
# /etc/resolv.conf and seeing this text, you have followed the symlink.
#
# This is a dynamic resolv.conf file for connecting local clients to the
# internal DNS stub resolver of systemd-resolved. This file lists all
# configured search domains.
#
# Run "resolvectl status" to see details about the uplink DNS servers
# currently in use.
#
# Third party programs should typically not access this file directly, but only
# through the symlink at /etc/resolv.conf. To manage man:resolv.conf(5) in a
# different way, replace this symlink by a static file or a different symlink.
#
# See man:systemd-resolved.service(8) for details about the supported modes of
# operation for /etc/resolv.conf.

nameserver 127.0.0.53
options edns0 trust-ad
search .
```

4. Update the Ubuntu package list (if required) using:

```
sudo apt update
```

5. Install the OpenSSH Server package using:

```
sudo apt install openssh-server -y
```

6. Enable and start the SSH service using:

7. `sudo systemctl enable ssh`

```
sudo systemctl start ssh
```

8. Check whether the SSH service is running successfully using:

```
sudo systemctl status
```

```
Processing triggers for dmw (0.30.2-9ubuntu1) ...
vboxuser@Ubuntu:~/Desktop$ sudo systemctl status ssh
○ ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; disabled; preset: enabled)
   Active: inactive (dead)
   TriggeredBy: ● ssh.socket
     Docs: man:sshd(8)
          man:sshd_config(5)
```

9. Find the IP address of the Ubuntu system using:

```
Hostname -I
```

10. Establish an SSH connection to the Ubuntu system using:

```
ssh <username>@<IP_address>
```



```

vboxuser@Ubuntu:~/Desktop$ hostname -I
10.0.2.15 10.20.20.1 fd17:625c:f037:2:a00:27ff:fe5c:c094
vboxuser@Ubuntu:~/Desktop$ ssh vboxuser@10.0.2.15
The authenticity of host '10.0.2.15 (10.0.2.15)' can't be established.
ED25519 key fingerprint is: SHA256:DPutENGud479D6pbIQjtRgFAhR1uP74ZPyEInGKGpbA
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.0.2.15' (ED25519) to the list of known hosts.
vboxuser@10.0.2.15's password:
Welcome to Ubuntu 26.04 LTS (GNU/Linux 7.0.0-28-generic x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

*** System restart required ***

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

```

11. Verify the successful SSH login by checking the "Welcome to Ubuntu" message displayed on the terminal.

CONCLUSION:

The Ubuntu system was successfully configured and verified for network connectivity. The OpenSSH Server was installed and enabled, and the SSH service was confirmed to be running. The system's IP address was obtained, and a successful SSH login was performed. Thus, secure remote access to the Ubuntu system was successfully established and verified.

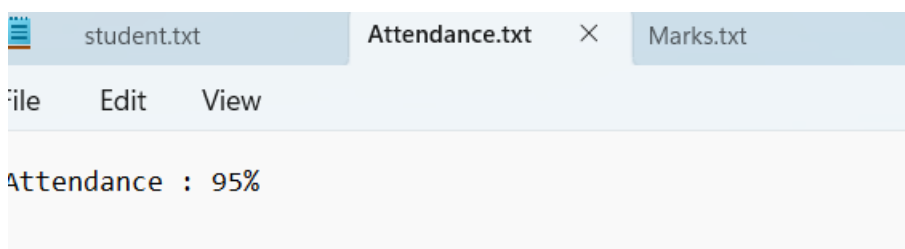
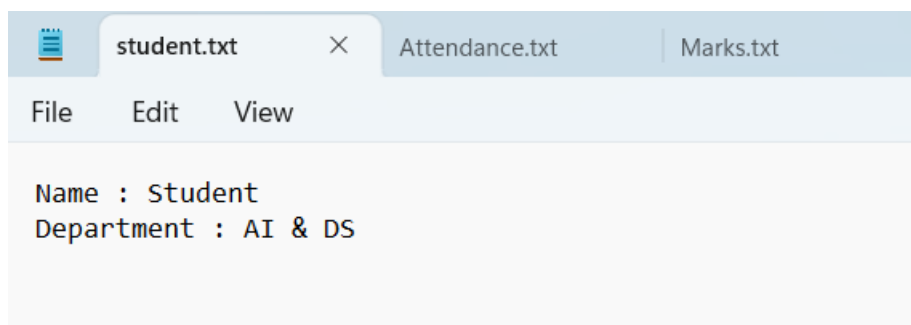
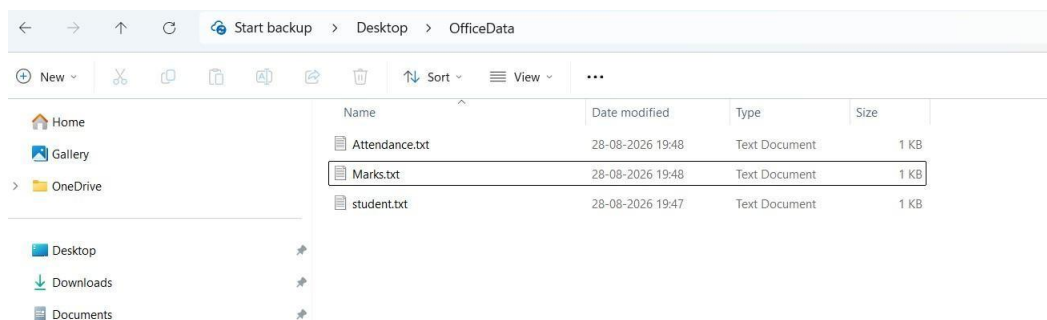
EXP NO 5	CLOUD MIGRATION – BACKUP SETUP USING GOOGLE DRIVE
Date: 14.08.2026	

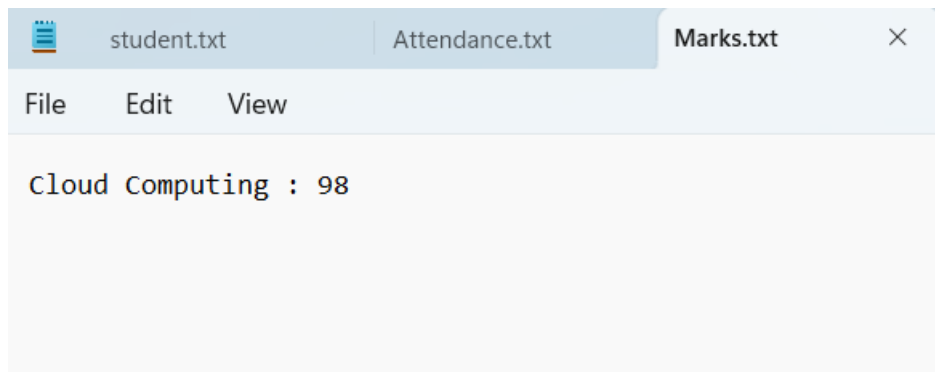
AIM

To create a cloud backup system by storing important files in Google Drive, enabling automatic synchronization, and verifying successful backup completion.

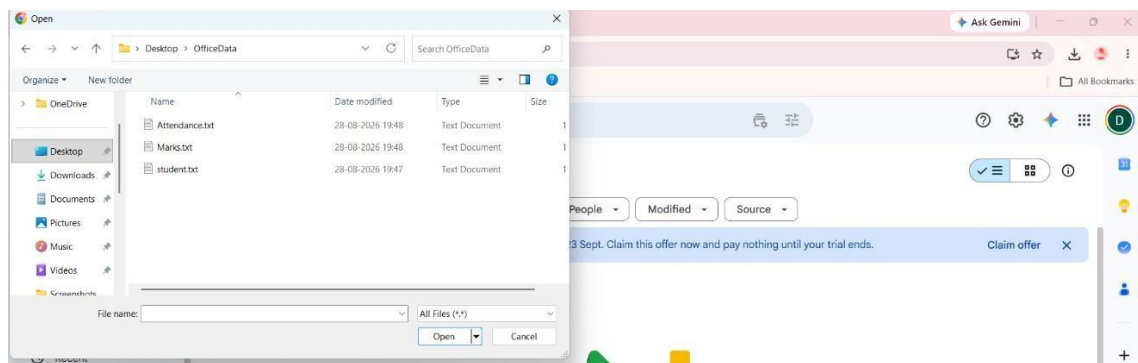
PROCEDURE

1. Create a folder named **OfficeData** on the Desktop and create three text files named **Student.txt**, **Attendance.txt**, and **Marks.txt** inside it. Add sample data to each file and save them.

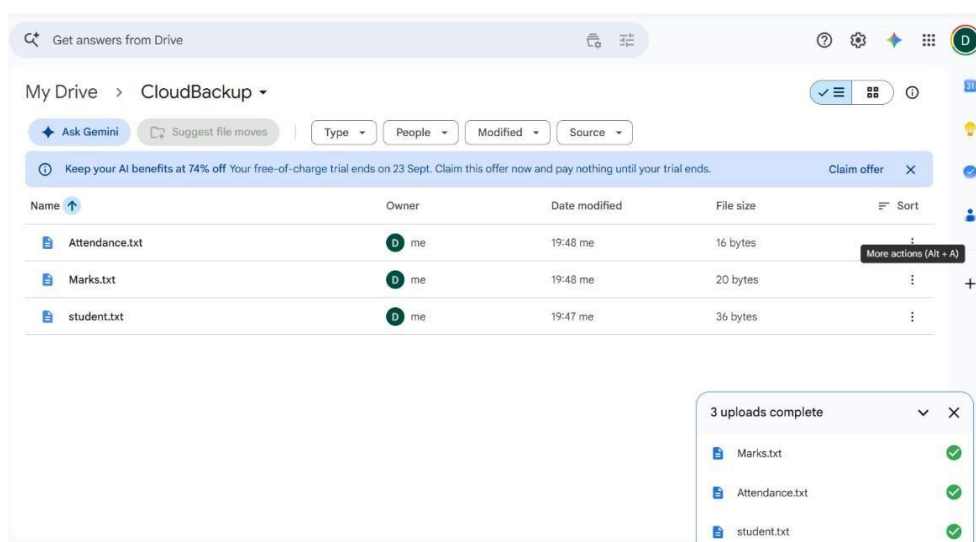




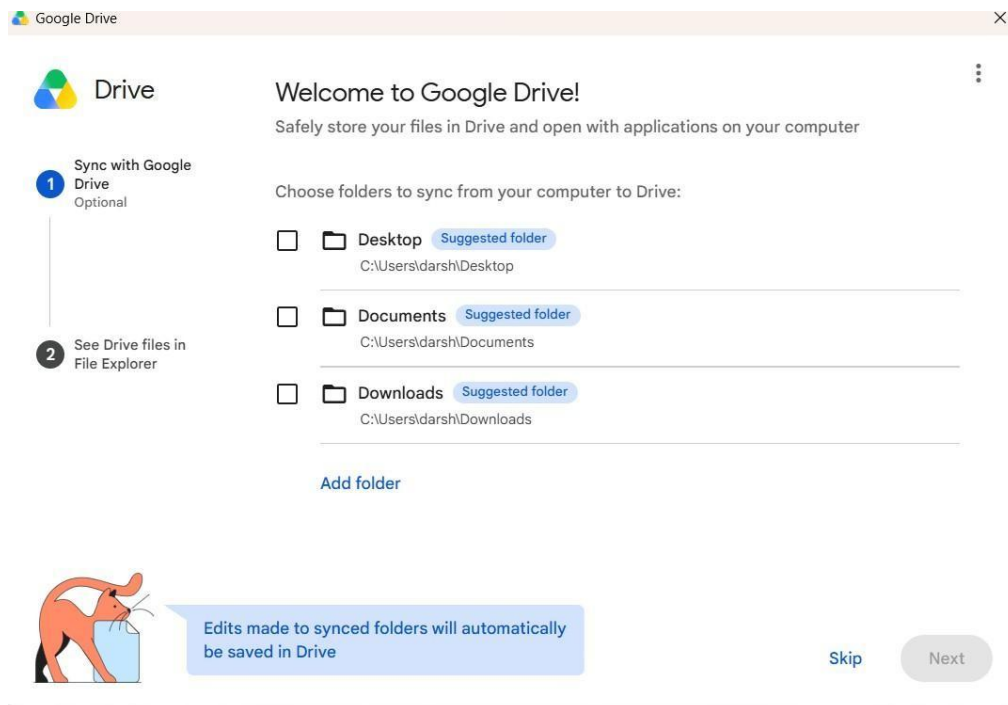
2. Open **Google Drive** using a web browser and log in with a valid Google account.
3. Create a new folder named **CloudBackup** in Google Drive.
4. Open the **CloudBackup** folder and select + **New** → **File Upload**.
5. Select **Student.txt**, **Attendance.txt**, and **Marks.txt** from the local **OfficeData** folder and upload them to Google Drive.



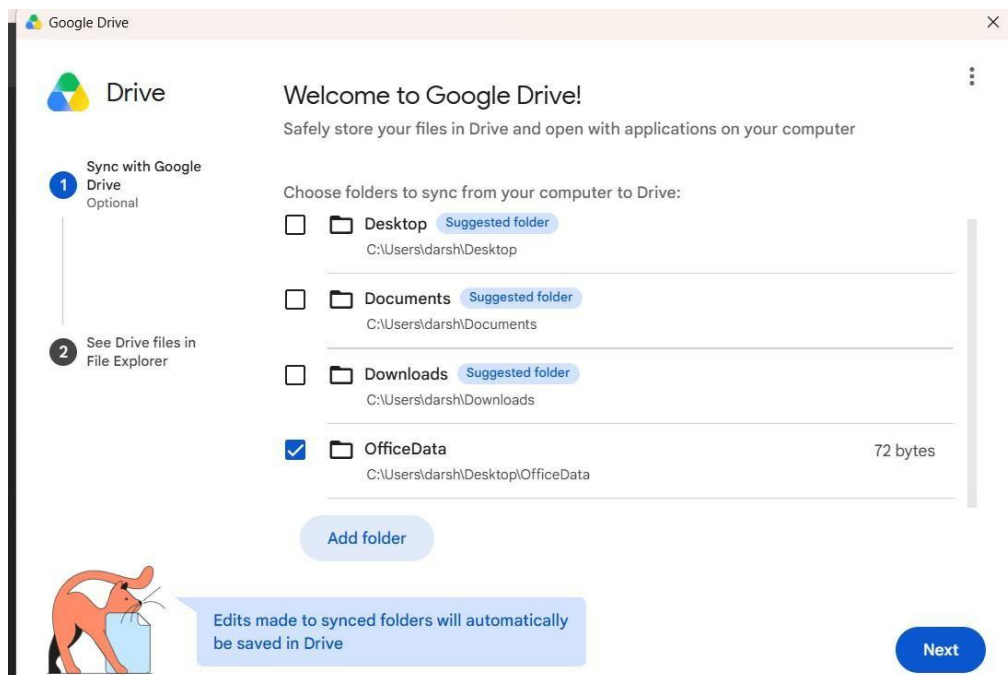
6. Verify that all three files are successfully available inside the **CloudBackup** folder in Google Drive.



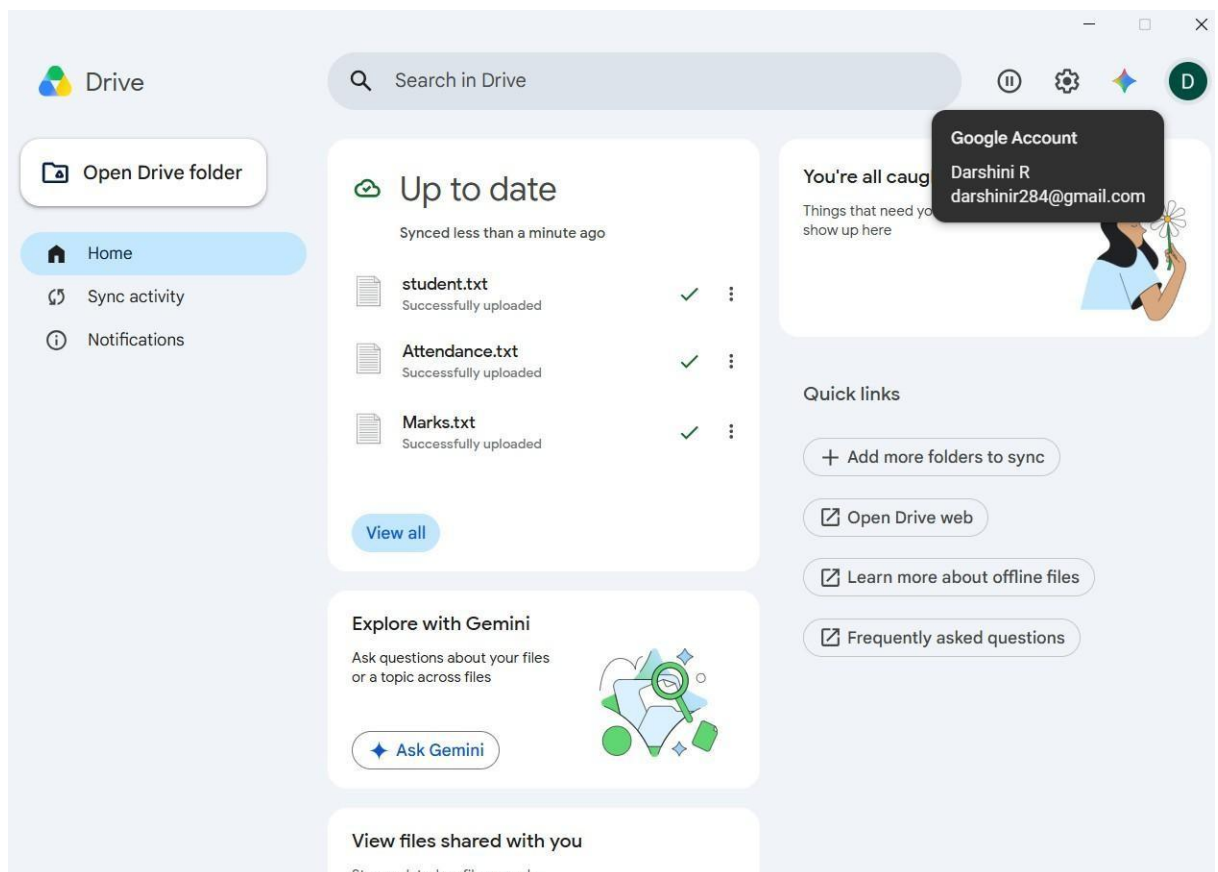
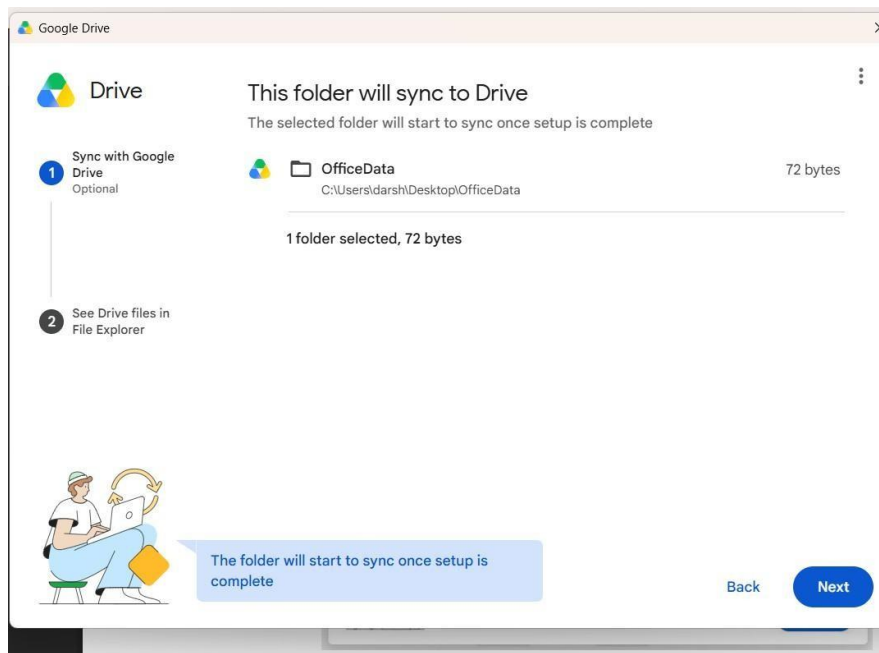
- Download and install **Google Drive for Desktop** on the Windows computer and sign in using the same Google account.



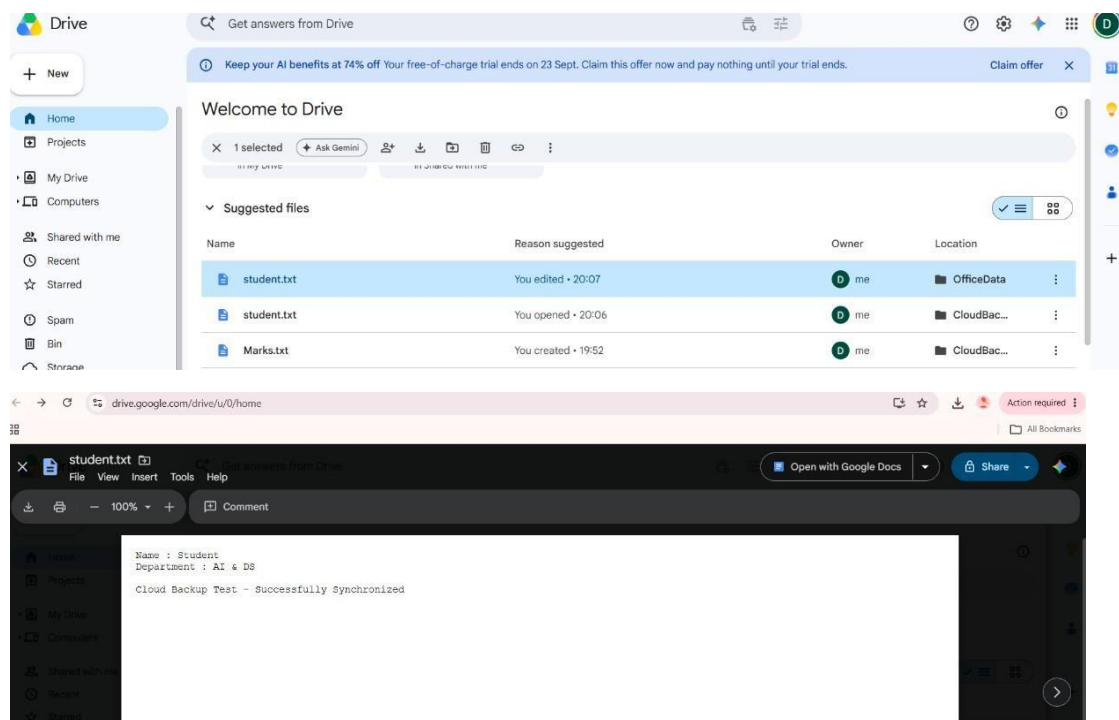
- Open **Google Drive for Desktop settings/preferences** and select the option to add a folder from the computer.
- Select the **OfficeData** folder located on the Desktop and configure it for synchronization/backup with Google Drive.



10. Verify that the files from the **OfficeData** folder are synchronized and available through Google Drive.



11. Open the local **Student.txt** file and add the following line at the end: **“Cloud Backup Test - Successfully Synchronized”**. Save the file.
12. Wait for the synchronization process to complete and verify that Google Drive for Desktop shows the files as synchronized or **Up to date**.
13. Refresh Google Drive in the browser, open the synchronized **Student.txt** file, and verify that the newly added line is present, confirming successful automatic cloud backup.



CONCLUSION

Thus, the cloud backup system was successfully implemented using **Google Drive**. The local files were migrated to cloud storage, automatic synchronization was enabled using Google Drive for Desktop, and changes made to the local files were successfully reflected in the cloud, demonstrating an effective cloud backup mechanism.

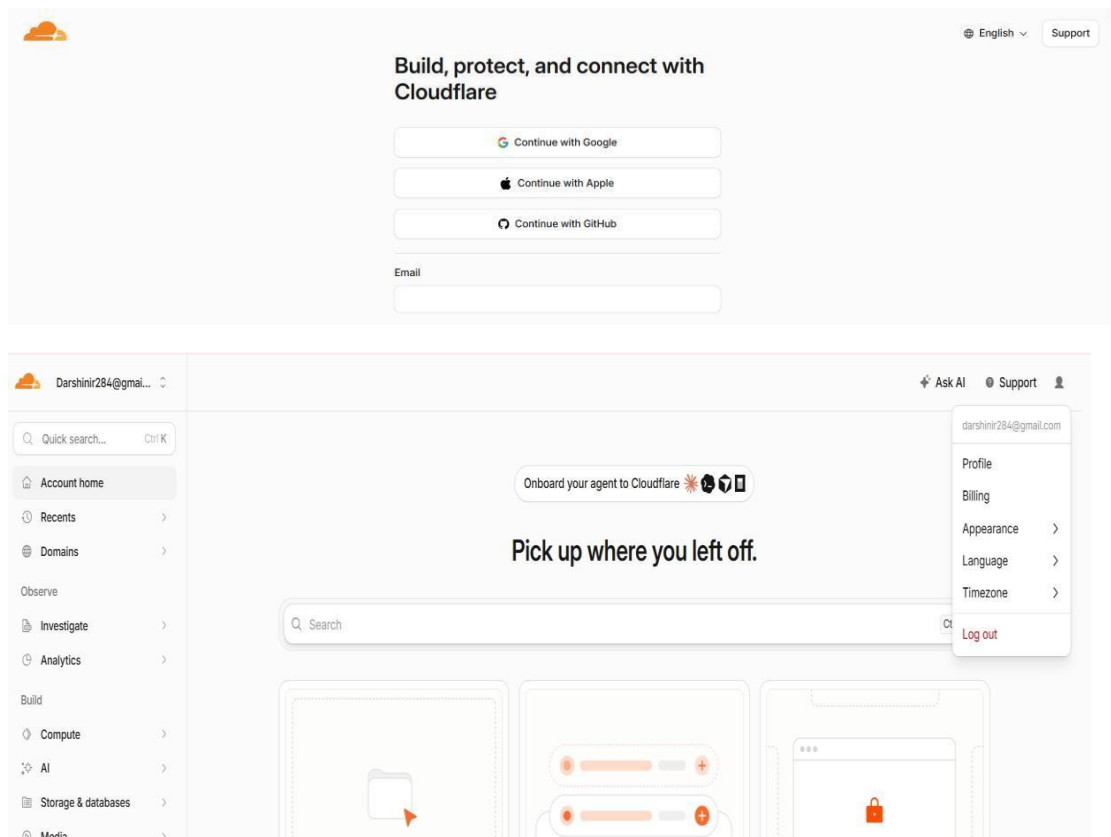
EXP NO 6	CDN SERVICE SELECTION AND CONFIGURATION USING CLOUDFLARE
Date:21.08.2026	

AIM

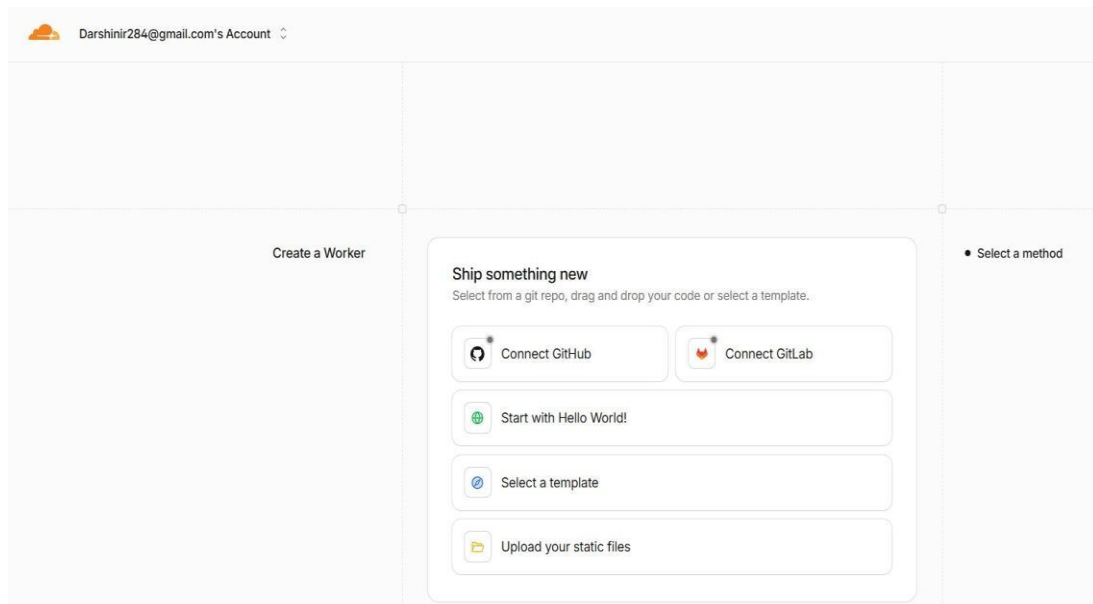
To select a Content Delivery Network (CDN) provider, deploy a simple web application using Cloudflare Workers, access the application through Cloudflare's edge network, and verify the HTTP response and Cloudflare-specific headers using browser Developer Tools.

PROCEDURE

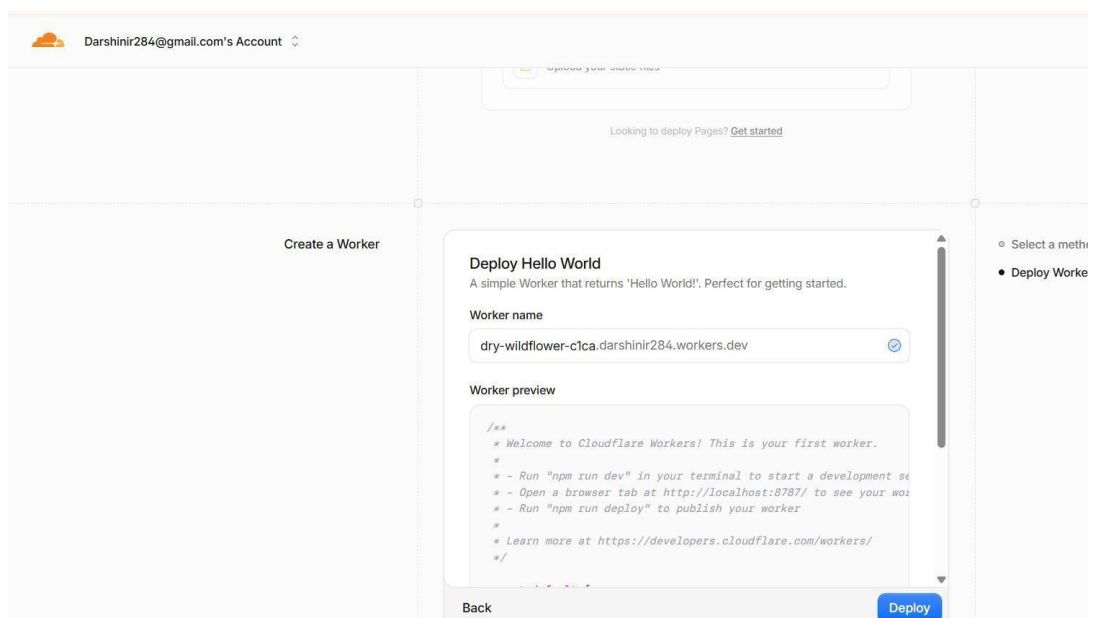
1. **Select the CDN/Edge Provider:** Cloudflare was selected as the CDN and edge computing provider because it provides CDN, security, DNS, and edge computing services.
2. **Create/Login to Cloudflare:** Open the Cloudflare website and create a new account or log in to an existing Cloudflare account. No domain or DNS configuration is required for this experiment.

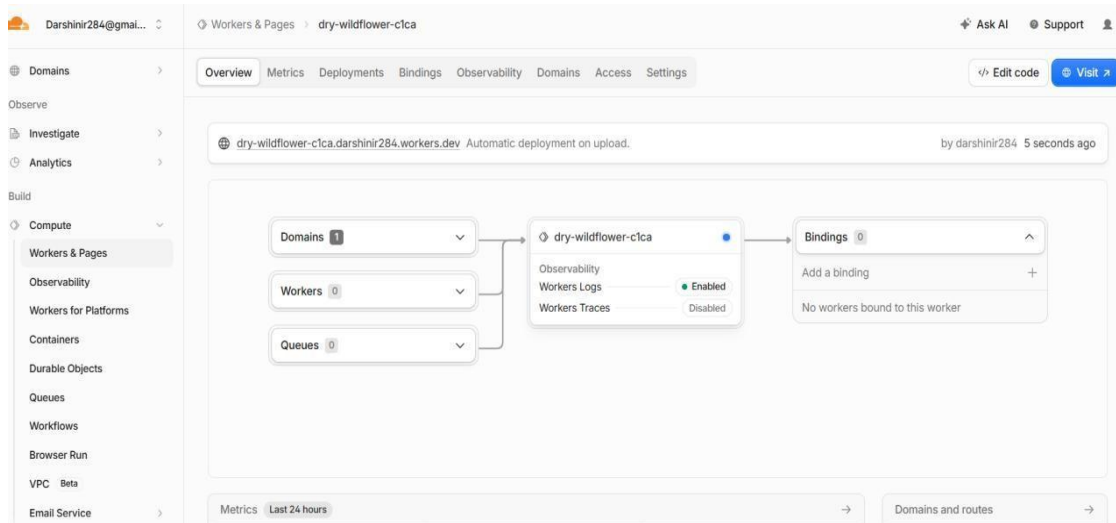


3. **Open Workers & Pages:** From the Cloudflare dashboard, navigate to **Workers & Pages** and select the option to create a new application.
4. **Create a Worker:** Select **Start with Hello World!** to create a simple Cloudflare Worker without using GitHub, GitLab, or external hosting services.

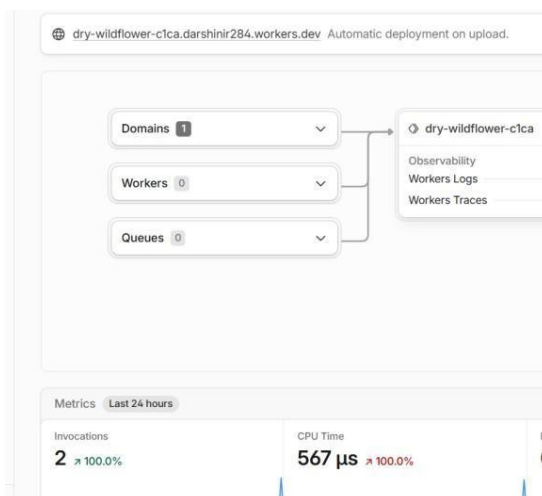
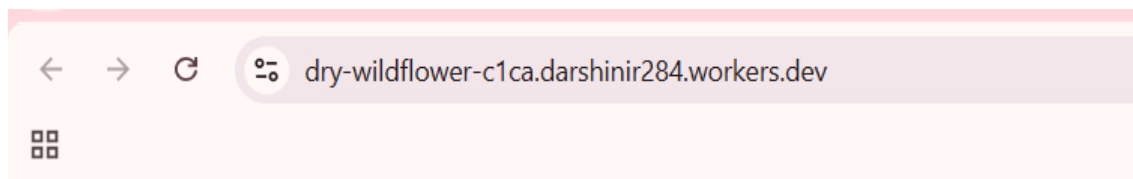


5. **Configure the Worker:** Keep the automatically generated Worker name and verify that the default Worker code returns the response Hello World!.
6. **Deploy the Worker:** Click **Deploy** and wait for Cloudflare to complete the deployment. A public workers.dev URL is generated for accessing the Worker.

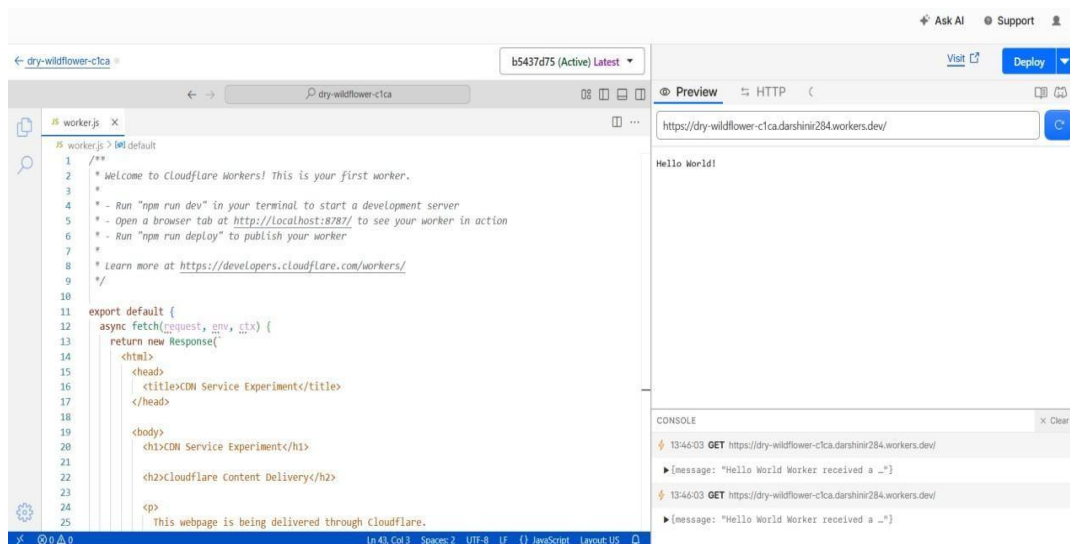




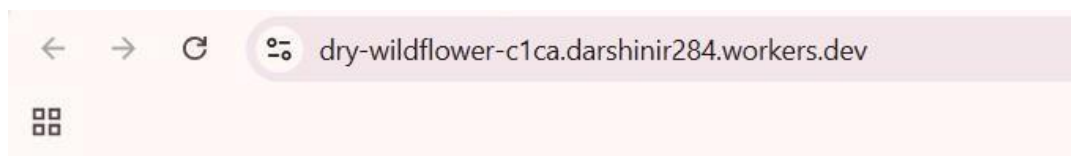
7. **Test the Deployment:** Click **Visit** or open the generated workers.dev URL in a browser. Verify that the page displays **Hello World!**, confirming that the Worker has been successfully deployed.



8. **Modify the Worker:** Open **Edit Code** and replace the default code with a simple HTML webpage containing the title **CDN Service Experiment**, information about Cloudflare content delivery, and the deployment status.



9. **Deploy the Webpage:** Save and deploy the modified Worker. Open the Worker URL again using **Visit** and verify that the newly created CDN experiment webpage is displayed correctly.



CDN Service Experiment

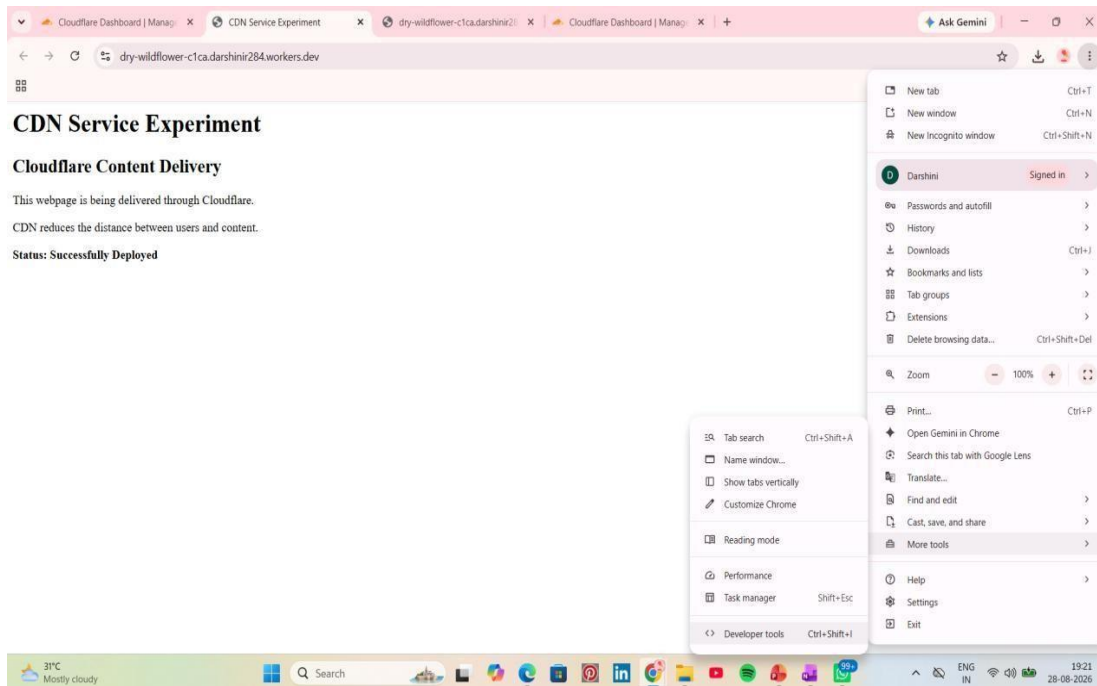
Cloudflare Content Delivery

This webpage is being delivered through Cloudflare.

CDN reduces the distance between users and content.

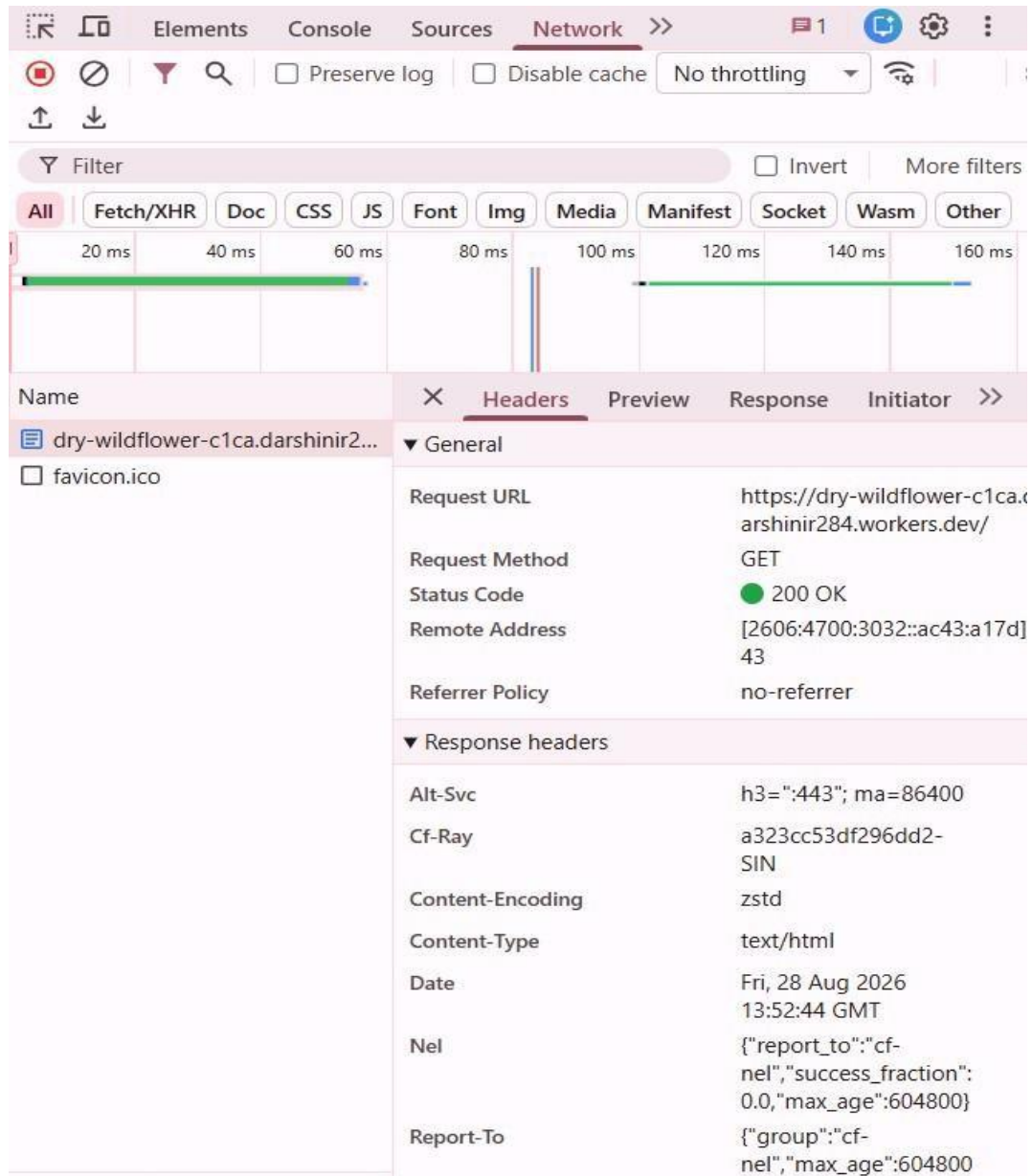
Status: Successfully Deployed

10. **Open Browser Developer Tools:** Open the Worker webpage in Google Chrome and open Developer Tools using **Ctrl + Shift + I**. Select the **Network** tab and reload the webpage using **Ctrl + R**.



11. **Inspect the HTTP Request:** Select the request corresponding to the workers.dev URL and open the **Headers** section. Verify the **Request URL**, **Request Method: GET**, and **Status Code: 200 OK**.
12. **Verify Cloudflare Response Headers:** Under **Response Headers**, verify the presence of Cloudflare-specific headers such as **Server: cloudflare** and **CF-Ray**. Also verify that the **Content-Type** is **text/html**.

13. **Record the Results:** Capture screenshots of the successful Worker deployment, the webpage, and the Network → Headers section showing 200 OK, Server: cloudflare, CF-Ray, and Content-Type: text/html. These results confirm that the application was successfully deployed and accessed through Cloudflare's edge infrastructure.



CONCLUSION

Thus, a simple web application was successfully deployed using Cloudflare Workers and accessed through a public workers.dev URL. The application returned an HTTP status code of **200 OK**, and Cloudflare-specific response headers such as **Server: cloudflare** and **CF-Ray** were observed using browser Developer Tools. Hence, the experiment successfully demonstrated the deployment and delivery of a web application through Cloudflare's edge network.

